

EX: No 1

INSTALLING DATA ANALYSIS AND

DATE: 17/07/25

VISUALIZATION TOOLS

AIM:

TO write a step to install data analysis & visualization tool: R/Python/Tableau Public/Power BI.

PROCEDURE:

1. R:

* Download R:

visit the official R website (<https://cran.r-project.org/>) and download the installer for your operating system (windows, macos or linux).

* Install R by following the instructions provided in the installer.

2. Python:

* Download Python:

visit the official website (<https://www.python.org/downloads/>) and download the Python installer for your os (windows, macos or linux)

* Install python by running the installer and making sure to check the option to add python to your system's PATH during installation.

(i) INSTALL NUMPY WITH PIP:

Numpy (Numerical Python) is an open-

source core Python library for scientific computations.

It is a general purpose array and matrices processing package.

`pip install numpy.`

(ii) INSTALL JUPYTER LAB:

Install Jupyter lab with `pip`:

`pip install jupyter lab`
once, installed, launch jupyter lab with:

`jupyter -lab`

(iii) JUPYTER NOTEBOOK:

Install the classic Jupyter Notebook with:

`pip install notebook.`

To run the notebook:

`Jupyter notebook.`

(iv) INSTALL SCIPY:

Scipy is a Python library that is useful in solving many mathematical equations and algorithms. It is designed on the top of numpy library. scipy means scientific python.

`pip install scipy.`

(v) INSTALL PANDAS:

Pandas is a Python Package that provides fast, flexible and expressive data structures designed to make working with "relational" or "labeled" data both easy and intuitive.

`pip install pandas.`

(vi) Install MATPLOTLIB:

Matplotlib is a comprehensive library for creating, static, animated and interactive visualizations in python working with "relational" or "labeled" data both easy and intuitive.

`pip install matplotlib.`

3. Tableau Public:

* It is a web-based tool, so there's no installation required. Simply visit the tableau public website (<https://public.tableau.com/s/gallery>) and create an account to start using it.

4. Power BI:

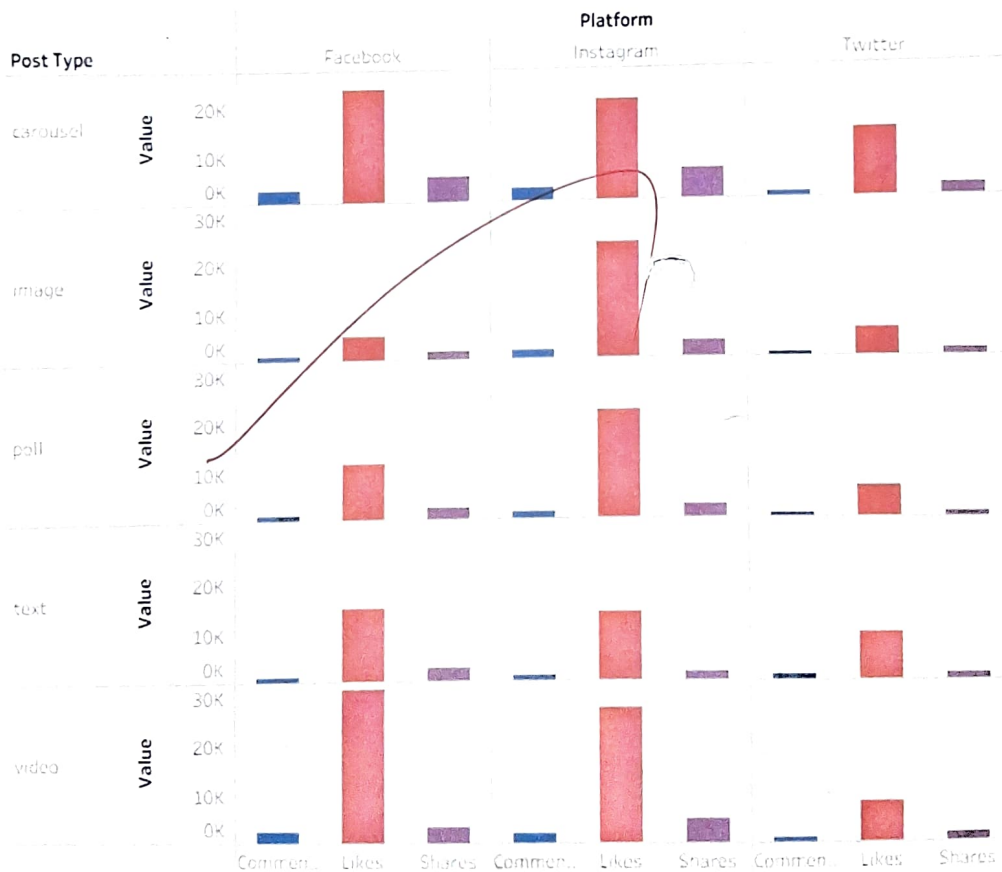
* Download Power BI Desktop:

Go to the official Power BI website (<https://powerbi.microsoft.com/en-us/desktop/>) and download Power BI Desktop.

* Install Power BI Desktop by running the installer.

	A	B	C	D	E	F	G	H	I	J
1	post_id	platform	post_type	post_time	likes	comment:	shares	post_day	sentiment_score	
2	1	Facebook	image	8/17/2023	2121	474	628	Thursday	positive	
3	2	Facebook	carousel	5/14/2023	3660	432	694	Sunday	neutral	
4	3	Instagram	poll	2/21/2023	4955	408	688	Tuesday	negative	
5	4	Twitter	image	11/16/202	1183	90	187	Thursday	negative	
6	5	Twitter	video	5/23/2023	3499	247	286	Tuesday	positive	
7	6	Instagram	carousel	#####	256	186	211	Friday	neutral	
8	7	Instagram	image	2/26/2023	1982	30	906	Sunday	positive	
9	8	Instagram	text	10/28/202	1274	45	216	Saturday	neutral	
10	9	Facebook	video	#####	317	249	221	Tuesday	neutral	
11	10	Twitter	carousel	#####	1878	225	438	Thursday	neutral	
12	11	Twitter	text	#####	1446	44	52	Friday	negative	
13	12	Twitter	video	6/15/2023	304	42	89	Thursday	positive	
14	13	Facebook	carousel	#####	45	78	375	Saturday	negative	
15	14	Twitter	text	9/30/2023	1333	144	123	Saturday	positive	

Sheet 1




```
[2]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

```
[3]: dict={"S.No": [1,2], "Student_Name": ["Ram", "Seetha"], "Student_Id": [11,12], "Marks": [85,95], "Grade": ['A', 'O']}
data=pd.DataFrame(dict)
data
```

```
[3]:
```

	S.No	Student_Name	Student_Id	Marks	Grade
0	1	Ram	11	85	A
1	2	Seetha	12	95	O

```
[4]: data.head()
```

```
[4]:
```

	S.No	Student_Name	Student_Id	Marks	Grade
0	1	Ram	11	85	A
1	2	Seetha	12	95	O

```
[5]: data.tail()
```

```
[5]:
```

	S.No	Student_Name	Student_Id	Marks	Grade
0	1	Ram	11	85	A
1	2	Seetha	12	95	O

```
[6]: data.describe()
```

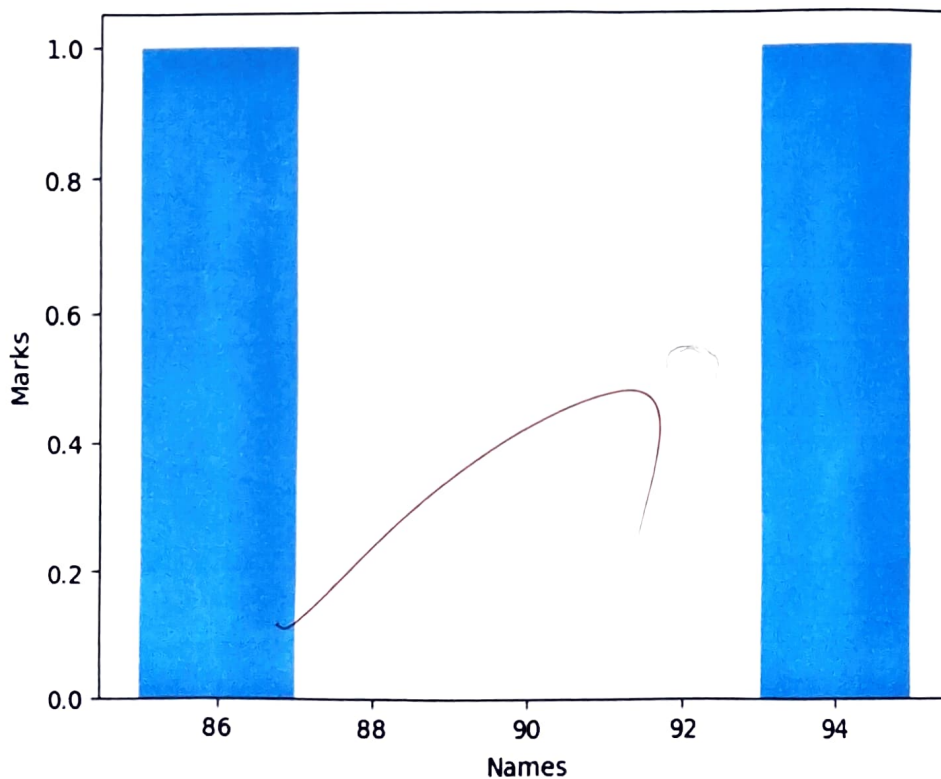
```
[6]:
```

	S.No	Student_Id	Marks
count	2.000000	2.000000	2.000000
mean	1.500000	11.500000	90.000000
std	0.707107	0.707107	7.071068
min	1.000000	11.000000	85.000000
25%	1.250000	11.250000	87.500000
50%	1.500000	11.500000	90.000000
75%	1.750000	11.750000	92.500000
max	2.000000	12.000000	95.000000

```
[7]: data.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2 entries, 0 to 1
Data columns (total 5 columns):
#   Column          Non-Null Count  Dtype
---  ---
0   S.No            2 non-null     int64
1   Student_Name    2 non-null     object
2   Student_Id      2 non-null     int64
3   Marks           2 non-null     int64
4   Grade           2 non-null     object
dtypes: int64(3), object(2)
memory usage: 212.0+ bytes
```

```
[8]: plt.hist(data["Marks"],bins=5)
plt.xlabel("Names")
plt.ylabel("Marks")
plt.show()
```



```
[1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

```
[2]: data=pd.read_csv("C:/Users/HP/Documents/Employee Details.csv")
data
```

```
[2]:
```

	S.No	Emp_id	Emp_name	Department	Salary
0	1	10	Alice	Sales	50,000
1	2	11	Mark	Finance	45,000
2	3	12	Bob	Marketing	40,000
3	4	13	John	HR	55,000

```
[3]: data.head()
```

```
[3]:
```

	S.No	Emp_id	Emp_name	Department	Salary
0	1	10	Alice	Sales	50,000
1	2	11	Mark	Finance	45,000
2	3	12	Bob	Marketing	40,000
3	4	13	John	HR	55,000

```
[4]: data.tail()
```

```
[4]:
```

	S.No	Emp_id	Emp_name	Department	Salary
0	1	10	Alice	Sales	50,000
1	2	11	Mark	Finance	45,000
2	3	12	Bob	Marketing	40,000
3	4	13	John	HR	55,000

```
[5]: data.describe()
```

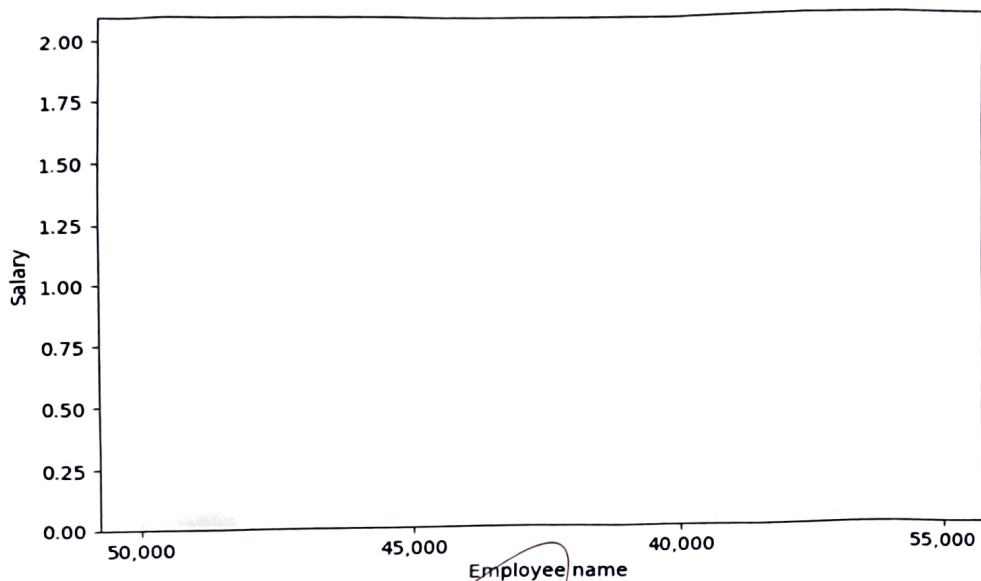
```
[5]:
```

	S.No	Emp_id
count	4.000000	4.000000
mean	2.500000	11.500000
std	1.290994	1.290994
min	1.000000	10.000000
25%	1.750000	10.750000
50%	2.500000	11.500000
75%	3.250000	12.250000
max	4.000000	13.000000

```
[7]: data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4 entries, 0 to 3
Data columns (total 5 columns):
#   Column          Non-Null Count  Dtype
---  -
0   S.No            4 non-null      int64
1   Emp_id          4 non-null      int64
2   Emp_name        4 non-null      object
3   Department      4 non-null      object
4   Salary          4 non-null      object
dtypes: int64(2), object(3)
memory usage: 292.0+ bytes
```

```
[9]: plt.figure(figsize=(8,5))  
plt.hist(data["Salary"],bins=3,color="pink",alpha=0.6)  
plt.xlabel("Employee name")  
plt.ylabel("Salary")  
plt.show()
```



RESULT:

16/04/2025
Thus the python program to install data analysis and visualization tools like R, python, Tableau, public, or power BI and their features were explored successfully.

EX: NO : 2

EXPLORATORY DATA ANALYSIS

DATE : 24/07/25

(EDA) ON WITH DATASETS

AIM :

TO perform Exploratory Data Analysis (EDA) on with datasets like email datasets.

PROCEDURE :

Exploratory Data Analysis (EDA) on email datasets involves importing the data, cleaning it, visualizing it, and extracting insights, here's a step by step guide on how to perform EDA on an email dataset using python and pandas.

1. Import necessary Libraries :

Import the required python libraries for data analysis and visualization.

2. Load Email Data :

Assuming you have a folder containing email files (e.g., eml files), you can use the email library to parse and extract the email contents.

3. Data cleaning :

Depending on your dataset, you may need to clean and preprocess the data. common cleaning steps include handling missing values, converting dates to datetime format and removing duplicates

4. Data Exploration!

Now you can start exploring the dataset using various techniques. Here are some common EDA tests:

BASIC STATISTICS:

Get summary statistics of the dataset

DISTRIBUTION OF DATES:

Visualize the distribution of email dates.

5. Word cloud for subject or message:

Create a wordcloud to visualize common words in email subjects or messages

6. Top senders and recipients:

Find the top email senders and recipients

Depending on your dataset, you can explore further, analyze sentiment, perform network analysis, or any other relevant analysis to gain insights from your email data.

```
[26]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from wordcloud import WordCloud
import warnings
warnings.simplefilter("ignore")
```

```
[27]: df=pd.read_csv("C:/Users/srmve/Downloads//spam_ham_dataset.csv")
```

```
[28]: df.head()
```

```
[28]:
```

	Unnamed: 0	label	text	label_num
0	605	ham	Subject: enron methanol ; meter # : 988291\\n...	0
1	2349	ham	Subject: hpl nom for january 9 , 2001\\n(see...	0
2	3624	ham	Subject: neon retreat\\nho ho ho , we ' re ar...	0
3	4685	spam	Subject: photoshop , windows , office . cheap ...	1
4	2030	ham	Subject: re : indian springs\\nthis deal is t...	0

```
[29]: df.tail()
```

```
[29]:
```

	Unnamed: 0	label	text	label_num
5166	1518	ham	Subject: put the 10 on the ft\\nthe transport...	0
5167	404	ham	Subject: 3 / 4 / 2000 and following noms\\nhp...	0
5168	2933	ham	Subject: calpine daily gas nomination\\n>\\n...	0
5169	1409	ham	Subject: industrial worksheets for august 2000...	0
5170	4807	spam	Subject: important online banking alert\\ndea...	1

```
[30]: df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5171 entries, 0 to 5170
Data columns (total 4 columns):
 #   Column        Non-Null Count  Dtype
---  -
 0   Unnamed: 0    5171 non-null   int64
 1   label         5171 non-null   object
 2   text          5171 non-null   object
 3   label_num     5171 non-null   int64
dtypes: int64(2), object(2)
memory usage: 161.7+ KB
```

```
[31]: df.describe()
```

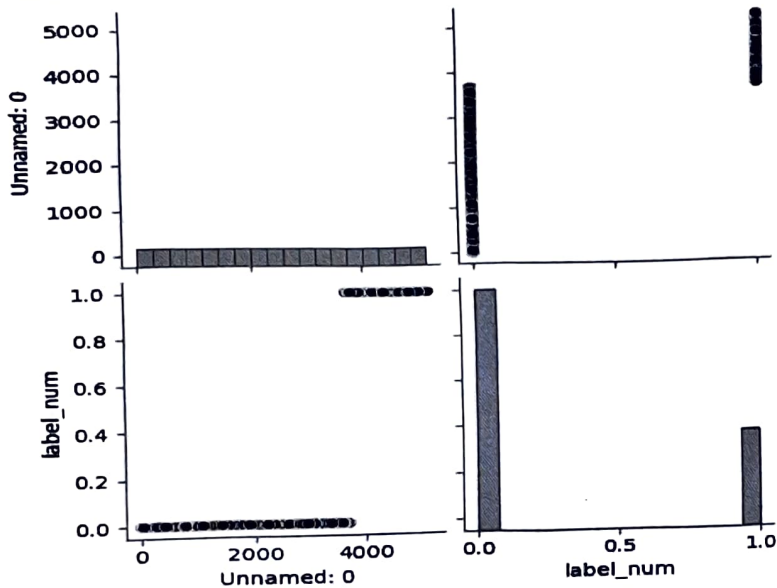
```
[31]:
```

	Unnamed: 0	label_num
count	5171.000000	5171.000000
mean	2585.000000	0.289886
std	1492.883452	0.453753
min	0.000000	0.000000
25%	1292.500000	0.000000
50%	2585.000000	0.000000
75%	3877.500000	1.000000
max	5170.000000	1.000000


```
[32]: df.isnull().sum()
```

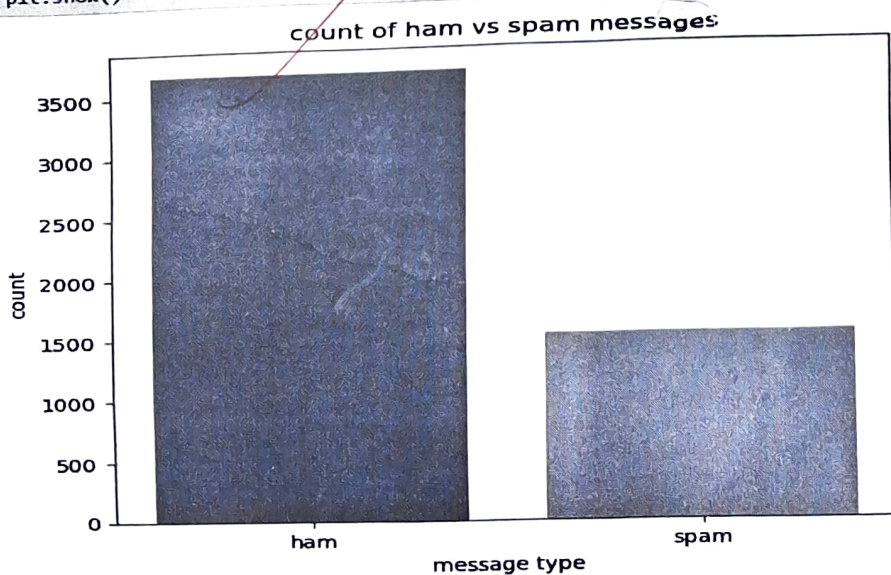
```
[32]: Unnamed: 0    0
      label        0
      text        0
      label_num    0
      dtype: int64
```

```
[33]: sns.pairplot(df)
      plt.show()
```



```
[10]:
```

```
import seaborn as sns
import matplotlib.pyplot as plt
sns.countplot(x='label',data=df)
plt.title("count of ham vs spam messages")
plt.xlabel("message type")
plt.ylabel("count")
plt.show()
```



EX: NO: 3

WORKING WITH NUMPY ARRAYS, PANDAS DATAFRAMES, BASIC PLOTS USING MATPLOTLIB.

DATE: 31/07/25

AIM: TO write the steps for working with Numpy arrays, pandas dataframes, Basic plots using matplotlib.

PROCEDURE:

NUMPY:

Numpy is a fundamental library for numerical computing in python. It provides support for multi-dimensional arrays and various mathematical functions.

```
[2]: import numpy as np

[3]: arr=np.array([1,7,3,8])
arr

[3]: array([1, 7, 3, 8])

[4]: sum=np.sum(arr)
sum

[4]: 19

[5]: mean=np.mean(arr)
mean

[5]: 4.75

[6]: sqrt=np.sqrt(arr)
sqrt

[6]: array([1.          , 2.64575131, 1.73205081, 2.82842712])

[7]: exponential=np.exp(arr)
exponential

[7]: array([2.71828183e+00, 1.09663316e+03, 2.00855369e+01, 2.98095799e+03])

[8]: first_element=arr[0]
first_element

[8]: 1

[9]: sub_array=arr[2:6]
sub_array

[9]: array([3, 8])

[10]: combined_array=np.concatenate([arr,sub_array])
combined_array

[10]: array([1, 7, 3, 8, 3, 8])

[11]: arr1=np.array([3,6,9,2])
arr1

[11]: array([3, 6, 9, 2])

[12]: combined_array1=np.concatenate([arr,arr1])
combined_array1

[12]: array([1, 7, 3, 8, 3, 6, 9, 2])
```

PANDAS:

Pandas is a powerful library for data manipulation and analysis. you can install pandas using pip:

Pip install Pandas.

PROGRAM:

```
[13]: import pandas as pd

[14]: data={'Person':['James', 'Amenda', 'Bobby', 'john'], 'Age':[28,30,27,31], 'Salary':[50000,45000,30000,55000]}
data
```

```
[15]: df=pd.DataFrame(data)
df

[16]:
```

	Person	Age	Salary
0	James	28	50000
1	Amenda	30	45000
2	Bobby	27	30000
3	john	31	55000

```
[16]: df['Person']

[16]:
```

0	James
1	Amenda
2	Bobby
3	john

Name: Person, dtype: object

```
[17]: df['Age']

[17]:
```

0	28
1	30
2	27
3	31

Name: Age, dtype: int64

```
[18]: df[['Person', 'Salary']]

[18]:
```

	Person	Salary
0	James	50000
1	Amenda	45000
2	Bobby	30000
3	john	55000

```
[19]: df[['Person', 'Age', 'Salary']]

[19]:
```

	Person	Age	Salary
0	James	28	50000
1	Amenda	30	45000
2	Bobby	27	30000
3	john	31	55000

```
[20]: df['Department']=['Sales', 'Finance', 'HR', 'Marketing']
df['Department']

[20]:
```

0	Sales
1	Finance
2	HR
3	Marketing

Name: Department, dtype: object

```
[21]: df

[21]:
```

	Person	Age	Salary	Department
0	James	28	50000	Sales
1	Amenda	30	45000	Finance
2	Bobby	27	30000	HR
3	john	31	55000	Marketing

```
[22]: print("Person older than 30")
      older=(df[df['Age']>30])
      older
```

Person older than 30

```
[22]:
```

	Person	Age	Salary	Department
3	john	31	55000	Marketing

```
[23]: print("Persons with high package")
      high=(df[df['Salary']>45000])
      high
```

Persons with high package

```
[23]:
```

	Person	Age	Salary	Department
0	James	28	50000	Sales
3	john	31	55000	Marketing

```
[24]: print("Sorting by 'Age' in ascending order:")
      sorted_Age=(df.sort_values(by='Age',ascending=True))
      sorted_Age
```

Sorting by 'Age' in ascending order:

```
[24]:
```

	Person	Age	Salary	Department
2	Bobby	27	30000	HR
0	James	28	50000	Sales
1	Amenda	30	45000	Finance
3	john	31	55000	Marketing

```
[25]: print("Sorting by 'Age' in descending order:")
      sorted_Age=(df.sort_values(by='Age',ascending=False))
      sorted_Age
```

Sorting by 'Age' in descending order:

```
[25]:
```

	Person	Age	Salary	Department
3	john	31	55000	Marketing
1	Amenda	30	45000	Finance
0	James	28	50000	Sales
2	Bobby	27	30000	HR

```
[26]: print("Sorting by 'Salary' in ascending order:")
      sorted_Salary=(df.sort_values(by='Salary',ascending=True))
      sorted_Salary
```

Sorting by 'Salary' in ascending order:

```
[26]:
```

	Person	Age	Salary	Department
2	Bobby	27	30000	HR
1	Amenda	30	45000	Finance
0	James	28	50000	Sales
3	john	31	55000	Marketing

```
[27]: Average_Age=df['Age'].mean()
      Average_Age
```

```
[27]: 29.0
```

```
[28]: Average_Salary=df['Salary'].mean()
Average_Salary

[28]: 45000.0

[29]: df.groupby('Person').groups

[29]: {'Amenda': [1], 'Bobby': [2], 'James': [0], 'John': [3]}

[30]: df.groupby(['Person', 'Age']).groups

[30]: {('Amenda', 30): [1], ('Bobby', 27): [2], ('James', 28): [0], ('John', 31): [3]}

[31]: grouped_data=df.groupby(['Person'])['Age'].sum()
print("Average age of person:")
print(grouped_data)

Average age of person:
Person
Amenda    30
Bobby     27
James     28
John      31
Name: Age, dtype: int64

[32]: grouped_data1=df.groupby('Person')['Age'].mean()
print(grouped_data1)

Person
Amenda    30.0
Bobby     27.0
James     28.0
John      31.0
Name: Age, dtype: float64

[33]: df['Age_Squared']=df['Age'].apply(lambda x: x**2)
df['Age_Squared']

[33]: 0    784
      1    900
      2    729
      3    961
      Name: Age_Squared, dtype: int64

[34]: df

[34]:
```

	Person	Age	Salary	Department	Age_Squared
0	James	28	50000	Sales	784
1	Amenda	30	45000	Finance	900
2	Bobby	27	30000	HR	729
3	John	31	55000	Marketing	961

```

[35]: df=df.drop(columns=['Age_Squared'])
df

[35]:
```

	Person	Age	Salary	Department
0	James	28	50000	Sales
1	Amenda	30	45000	Finance
2	Bobby	27	30000	HR
3	John	31	55000	Marketing

```

[85]: df.to_csv('output.csv',index=False)

[86]: new_df=pd.read_csv('output.csv')
new_df

```


MATPLOTLIB :

matplotlib is a popular library for creating static, animated or interactive plots and graphs.

Install matplotlib using pip:

Pip install matplotlib

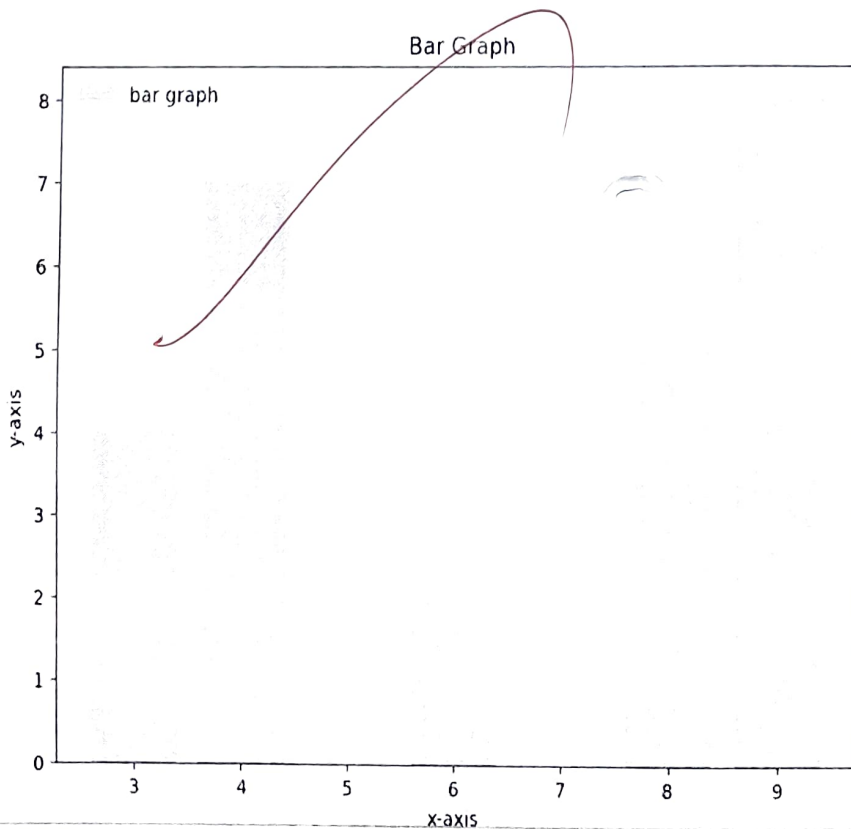
PROGRAM:

```
[10]: import matplotlib.pyplot as plt
```

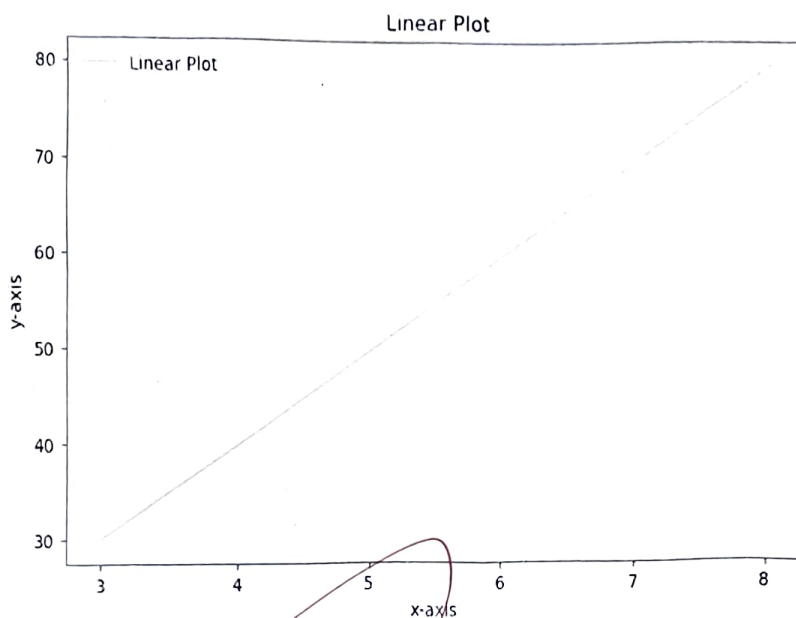
```
[11]: x=[3,6,4,8,9]
```

```
y=[4,2,7,5,8]
```

```
[18]: plt.figure(figsize=(8,6))
plt.bar(x,y,label='bar graph',color='violet',alpha = 0.7)
plt.title('Bar Graph')
plt.xlabel('x-axis')
plt.ylabel('y-axis')
plt.legend()
plt.grid(False)
plt.show()
```



```
plt.figure(figsize=(8,5))
plt.plot(x,y,label='Linear Plot',color='green',alpha=0.5)
plt.title('Linear Plot')
plt.xlabel('x-axis')
plt.ylabel('y-axis')
plt.legend()
plt.grid(False)
plt.show()
```



RESULT:

Thus the above working with numpy, pandas and matplotlib has been completed successfully.

EX: NO: 4

EXPLORING VARIOUS VARIABLE AND ROW FILTERS

DATE: 07/08/25

IN R FOR CLEANING DATA

AIM:

TO exploring various variable and row filters in R for cleaning data.

DATA PREPARATION AND CLEANING:

First lets load a sample dataset and then explore various variables and row filters to clean the data.

Loading and printing the sample dataset.

```
> data<-read.csv("C:/Users/srmve/Downloads/Fruit Sales Data (1).csv")
> data
```

	Fruit.Name	Weight.in.Kg	Total.Cost	Purchased
1	Apple	44	72000	No
2	Mango	27	48000	Yes
3	Orange	30	54000	No
4	Strawberry	38	61000	No
5	Orange	40	NA	Yes
6	Mango	35	58000	Yes
7	Apple	NA	52000	No
8	Strawberry	48	79000	Yes
9	Strawberry	50	83000	No
10	Apple	37	67000	Yes
11	Orange	47	63000	No
12	Mango	39	NA	Yes
13	Mango	46	76000	No
14	Orange	29	34000	No
15	Strawberry	NA	38500	Yes
16	Apple	24	38000	Yes

Variable filters:

```
> head(data)
Fruit.Name Weight.in.Kg Total.Cost Purchased
1 Apple 44 72000 No
2 Mango 27 48000 Yes
3 Orange 30 54000 No
4 Strawberry 38 61000 No
5 Orange 40 NA Yes
6 Mango 35 58000 Yes
```

```
> tail(data)
Fruit.Name Weight.in.Kg Total.Cost Purchased
11 Orange 47 63000 No
12 Mango 39 NA Yes
13 Mango 46 76000 No
14 Orange 29 34000 No
15 Strawberry NA 38500 Yes
16 Apple 24 38000 Yes
```

```
> summary(data)
Fruit.Name      Weight.in.Kg      Total.Cost
Length:16      Min.   :24.00      Min.   :34000
Class :character 1st Qu.:31.25      1st Qu.:49000
Mode :character  Median :38.50      Median :59500
                  Mean   :38.14      Mean   :58821
                  3rd Qu.:45.50      3rd Qu.:70750
                  Max.   :50.00      Max.   :83000
                  NA's   :2         NA's   :2
```

```
Purchased
Length:16
Class :character
Mode :character
```

```
> str(data)
'data.frame': 16 obs. of 4 variables:
 $ Fruit.Name : chr "Apple" "Mango" "Orange" "Strawberry" ...
 $ Weight.in.Kg: int 44 27 30 38 40 35 NA 48 50 37 ...
 $ Total.Cost : int 72000 48000 54000 61000 NA 58000 52000 79000 83000 6
 7000 ...
 $ Purchased : chr "No" "Yes" "No" "No" ...
```

```
> sapply(data, function(x) sum(is.na(x)))
Fruit.Name Weight.in.Kg Total.Cost Purchased
0 2 2 0
```



```
data$weight.in.Kg[is.na(data$weight.in.Kg)] <- mean(data$weight.in.Kg, na.rm = TRUE)
> data$weight.in.Kg
[1] 44.00000 27.00000 30.00000 38.00000 40.00000 35.00000 38.14286 48.000
00 50.00000 37.00000
[11] 47.00000 39.00000 46.00000 29.00000 38.14286 24.00000

> data$weight.in.Kg[is.na(data$weight.in.Kg)] <- mean(data$weight.in.Kg, n
a.rm = TRUE)
> data$weight.in.Kg
[1] 44.00000 27.00000 30.00000 38.00000 40.00000
[6] 35.00000 38.14286 48.00000 50.00000 37.00000
[11] 47.00000 39.00000 46.00000 29.00000 38.14286
[16] 24.00000

> data$Total.Cost[is.na(data$Total.Cost)] <- mean(data$Total.Cost, na.rm =
TRUE)
> data$Total.Cost
[1] 72000.00 48000.00 54000.00 61000.00 58821.43
[6] 58000.00 52000.00 79000.00 83000.00 67000.00
[11] 63000.00 58821.43 76000.00 34000.00 38500.00
[16] 38000.00
```

Row filters:

```
> data <- data[!duplicated(data), ]
> data
  Fruit.Name weight.in.Kg Total.Cost Purchased
1      Apple    44.00000    72000.00        No
2      Mango    27.00000    48000.00        Yes
3      Orange    30.00000    54000.00        No
4 Strawberry    38.00000    61000.00        No
5      Orange    40.00000    58821.43        Yes
6      Mango    35.00000    58000.00        Yes
7      Apple    38.14286    52000.00        No
8 Strawberry    48.00000    79000.00        Yes
9 Strawberry    50.00000    83000.00        No
10     Apple    37.00000    67000.00        Yes
11     Orange    47.00000    63000.00        No
12     Mango    39.00000    58821.43        Yes
13     Mango    46.00000    76000.00        No
14     Orange    29.00000    34000.00        No
15 Strawberry    38.14286    38500.00        Yes
16     Apple    24.00000    38000.00        Yes
```

```
> data[data$weight.in.Kg > 40, ]
  Fruit.Name weight.in.Kg Total.Cost Purchased
1      Apple    44.00000    72000.00        No
8 Strawberry    48.00000    79000.00        Yes
9 Strawberry    50.00000    83000.00        No
11     Orange    47.00000    63000.00        No
13     Mango    46.00000    76000.00        No
```

```
> data[data$weight.in.Kg > 40 & data$Total.Cost > 60000, ]
  Fruit.Name weight.in.Kg Total.Cost Purchased
1      Apple    44.00000    72000.00        No
8 Strawberry    48.00000    79000.00        Yes
9 Strawberry    50.00000    83000.00        No
11     Orange    47.00000    63000.00        No
13     Mango    46.00000    76000.00        No
```

```
> data[data$Weight.in.Kg > 40 | data$Total.Cost > 60000, ]
```

	Fruit.Name	Weight.in.Kg	Total.Cost	Purchased
1	Apple	44	72000	No
4	Strawberry	38	61000	No
8	Strawberry	48	79000	Yes
9	Strawberry	50	83000	No
10	Apple	37	67000	Yes
11	Orange	47	63000	No
13	Mango	46	76000	No

```
> data[, c("Weight.in.Kg", "Total.Cost")]
```

	Weight.in.Kg	Total.Cost
1	44.00000	72000.00
2	27.00000	48000.00
3	30.00000	54000.00
4	38.00000	61000.00
5	40.00000	58821.43
6	35.00000	58000.00
7	38.14286	52000.00
8	48.00000	79000.00
9	50.00000	83000.00
10	37.00000	67000.00
11	47.00000	63000.00
12	39.00000	58821.43
13	46.00000	76000.00
14	29.00000	34000.00
15	38.14286	38500.00
16	24.00000	38000.00

```
> outliers_weight <- boxplot.stats(data$Weight.in.Kg)$out
> outliers_weight
numeric(0)
```

```
> outliers_cost <- boxplot.stats(data$Total.Cost)$out
> outliers_cost
numeric(0)
```

```
> outliers_weight <- data[data$Weight.in.Kg < quantile(data$Weight.in.Kg,
0.25) - 1.5 * IQR(data$Weight.in.Kg) | data$Weight.in.Kg > quantile(data$W
eight.in.Kg, 0.75) + 1.5 * IQR(data$Weight.in.Kg), ]
> outliers_weight
[1] Fruit.Name Weight.in.Kg Total.Cost Purchased
<0 rows> (or 0-length row.names)
```

```
> outliers_cost <- data[data$Total.Cost < quantile(data$Total.Cost, 0.25)
- 1.5 * IQR(data$Total.Cost) | data$Total.Cost > quantile(data$Total.Cost,
0.75) + 1.5 * IQR(data$Total.Cost), ]
> outliers_cost
[1] Fruit.Name Weight.in.Kg Total.Cost Purchased
<0 rows> (or 0-length row.names)
```

```
> q1_weight <- quantile(data$Weight.in.Kg, 0.25)
```

```
> q3_weight <- quantile(data$Weight.in.Kg, 0.75)
```

```
> iqr_weight <- q3_weight - q1_weight
```

```
> q1_weight
25%
33.75
```

```
> q3_weight
75%
44.5
```

```

> iqr_weight
75%
10.75

> q1_cost <- quantile(data$Total.Cost, 0.25)
> q3_cost <- quantile(data$Total.Cost, 0.75)
> iqr_cost <- q3_cost - q1_cost

> q1_cost
25%
51000

> q3_cost
75%
68250

> iqr_cost
75%
17250

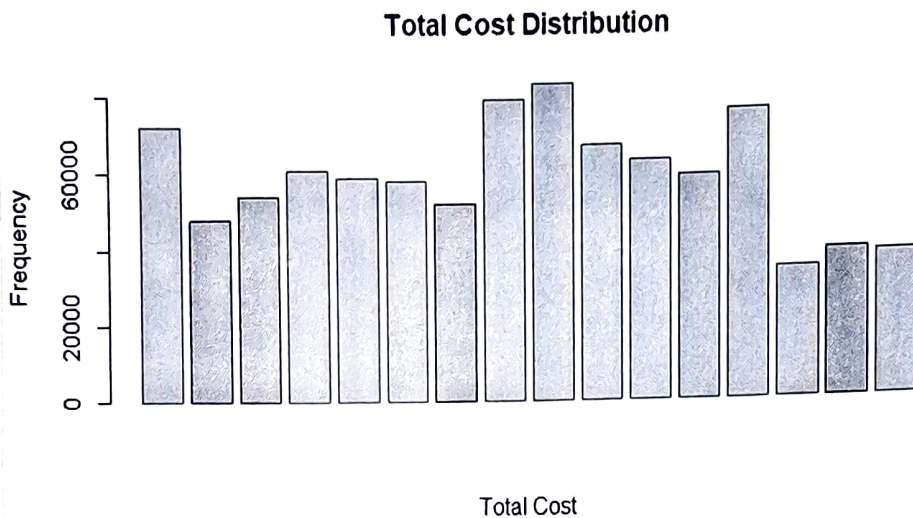
> outliers_weight <- data[data$Weight.in.Kg < (q1_weight - 1.5 * iqr_weight) | data$Weight.in.Kg > (q3_weight + 1.5 * iqr_weight), ]
> outliers_weight
[1] Fruit.Name Weight.in.Kg Total.Cost Purchased
<0 rows> (or 0-length row.names)

> outliers_cost <- data[data$Total.Cost < (q1_cost - 1.5 * iqr_cost) | data$Total.Cost > (q3_cost + 1.5 * iqr_cost), ]
> outliers_cost
[1] Fruit.Name Weight.in.Kg Total.Cost Purchased
<0 rows> (or 0-length row.names)

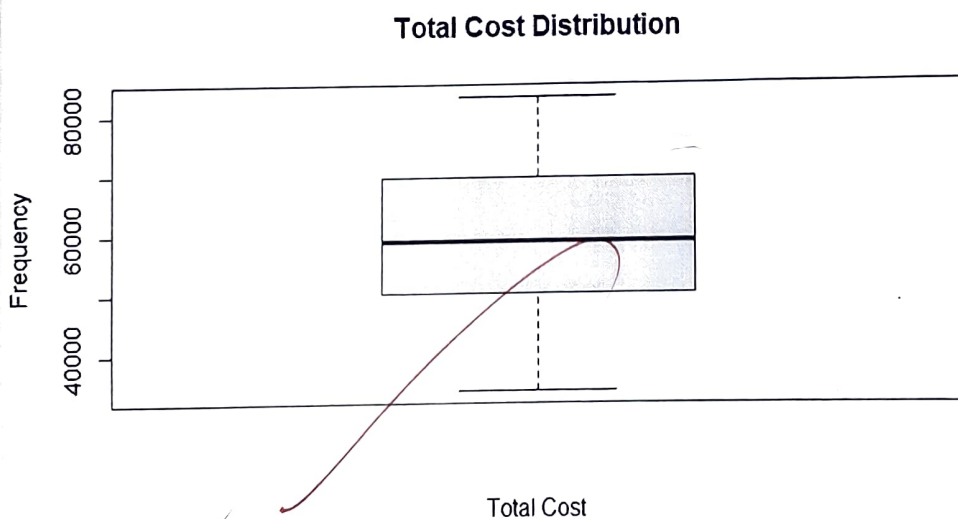
> data <- data[data$Weight.in.Kg >= (q1_weight - 1.5 * iqr_weight) & data$Weight.in.Kg <= (q3_weight + 1.5 * iqr_weight) & data$Total.Cost >= (q1_cost - 1.5 * iqr_cost) & data$Total.Cost <= (q3_cost + 1.5 * iqr_cost), ]
> data
  Fruit.Name weight.in.Kg Total.Cost Purchased
1      Apple    44.00000    72000.00        No
2      Mango    27.00000    48000.00        Yes
3      Orange    30.00000    54000.00        No
4  Strawberry    38.00000    61000.00        No
5      Orange    40.00000    58821.43       Yes
6      Mango    35.00000    58000.00       Yes
7      Apple    38.14286    52000.00        No
8  Strawberry    48.00000    79000.00       Yes
9  Strawberry    50.00000    83000.00        No
10     Apple    37.00000    67000.00       Yes
11     Orange    47.00000    63000.00        No
12     Mango    39.00000    58821.43       Yes
13     Mango    46.00000    76000.00        No
14     Orange    29.00000    34000.00        No
15  Strawberry    38.14286    38500.00       Yes
16     Apple    24.00000    38000.00       Yes

```

```
barplot(data$Total.Cost, main = "Total Cost Distribution", xlab = "Total C  
ost", ylab = "Frequency")
```



```
boxplot(data$Total.Cost, main = "Total Cost Distribution", xlab = "Total C  
ost", ylab = "Frequency")
```



RESULT :

Thus the above exploring various variable and row filters in R for cleaning data was successfully completed.

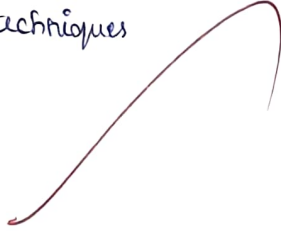
EX: NO : 5

DATE: 14/08/25

TIME SERIES ANALYSIS USING VARIOUS VISUALIZATION TECHNIQUES

AIM :

TO perform time series analysis and apply the various
visualization techniques



```
[1]: import pandas as pd
import matplotlib.pyplot as plt

[2]: data=pd.read_csv("C:/Users/srmve/Downloads/sales.csv")
data
```

```
[2]:
```

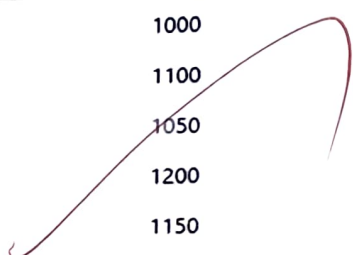
	Timestamp	Sales Amount (USD)
0	2000-01-01	1000
1	2000-02-01	1100
2	2000-03-01	1050
3	2000-04-01	1200
4	2000-05-01	1150
...	...	...
369	2030-10-01	19450
370	2030-11-01	19500
371	2030-12-01	19550
372	2031-01-01	19600
373	2031-02-01	19650

374 rows x 2 columns

```
[3]: data.head()
```

```
[3]:
```

	Timestamp	Sales Amount (USD)
0	2000-01-01	1000
1	2000-02-01	1100
2	2000-03-01	1050
3	2000-04-01	1200
4	2000-05-01	1150



```
[4]: data.tail()
```

```
[4]:
```

	Timestamp	Sales Amount (USD)
369	2030-10-01	19450
370	2030-11-01	19500
371	2030-12-01	19550
372	2031-01-01	19600
373	2031-02-01	19650

```
[5]: data.describe()
```

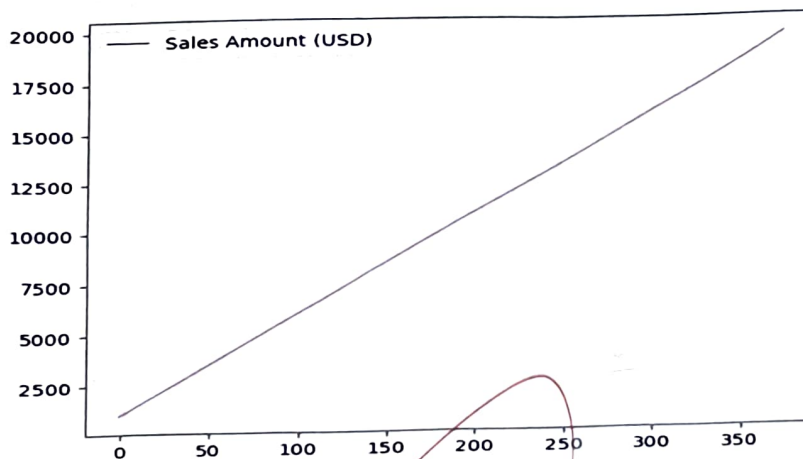
```
[5]:
```

Sales Amount (USD)	
count	374.000000
mean	10325.000000
std	5405.437078
min	1000.000000
25%	5662.500000
50%	10325.000000
75%	14987.500000
max	19650.000000

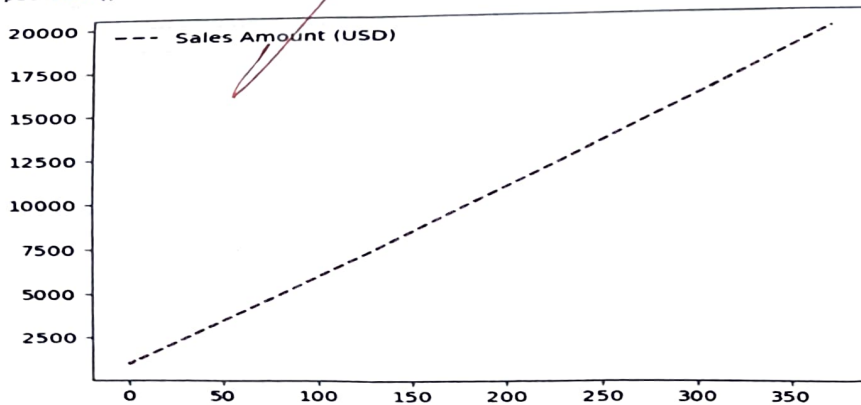
```
[6]: data.info()
```

```
<class 'pandas.core.frame.DataFrame'>  
RangeIndex: 374 entries, 0 to 373  
Data columns (total 2 columns):  
#   Column                Non-Null Count  Dtype    
---  ---                  
0   Timestamp              374 non-null   object   
1   Sales Amount (USD)     374 non-null   int64    
dtypes: int64(1), object(1)  
memory usage: 6.0+ KB
```

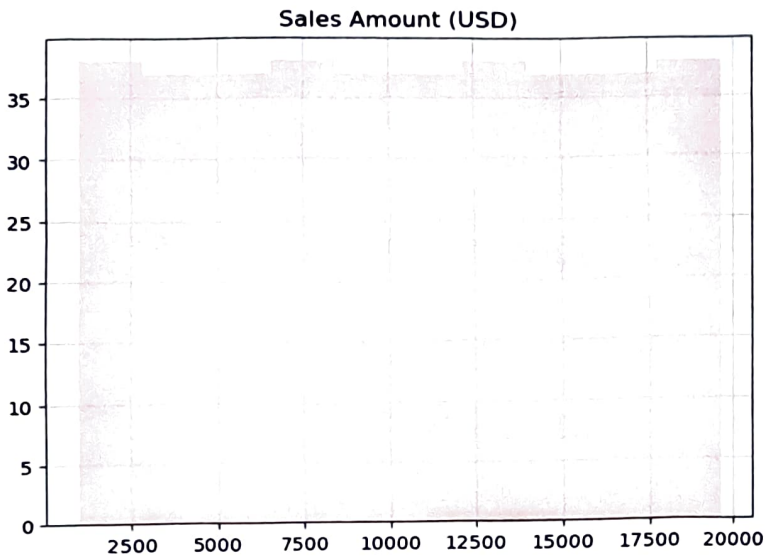
```
[7]: data.plot(color='purple',alpha=0.6)  
plt.show()
```



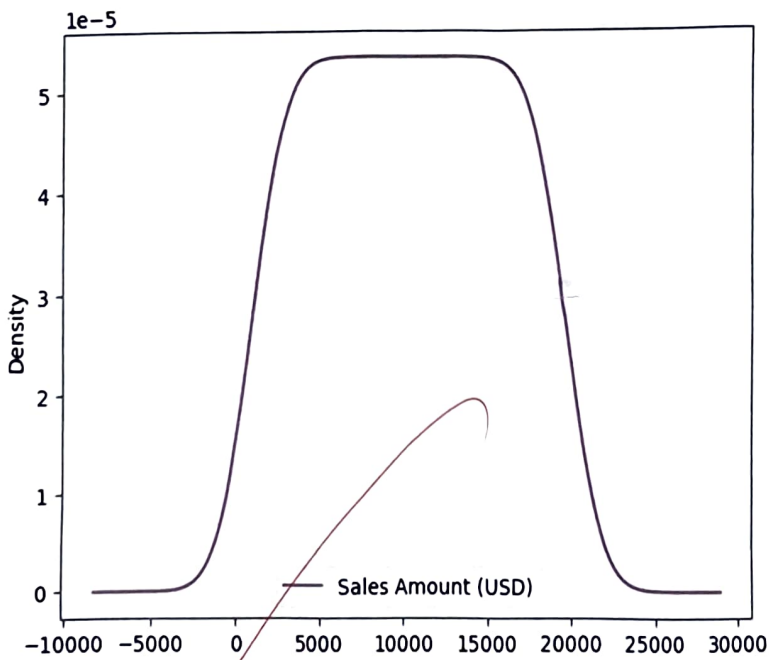
```
[8]: data.plot(style='--',color='purple')  
plt.show()
```



```
[28]: data.hist(color='pink')
plt.show()
```



```
[30]: data.plot(kind='kde',color='purple')
plt.show()
```



RESULT

Thus the above time series has been checked with various visualization techniques.

EX: NO: 6

DATA ANALYSIS AND REPRESENTATION

DATE: 21/08/25

ON MAP

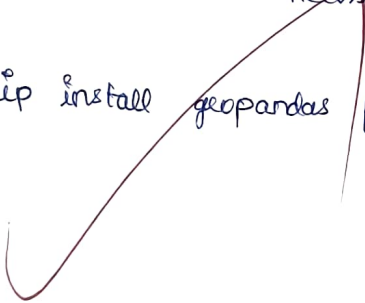
AIM:

TO write a program to perform data analysis and representation on a map using various map data sets with mouse rollover effect, user interaction etc...

PROCEDURE:

Make sure to install the necessary libraries

Pip install geopandas folium bokeh

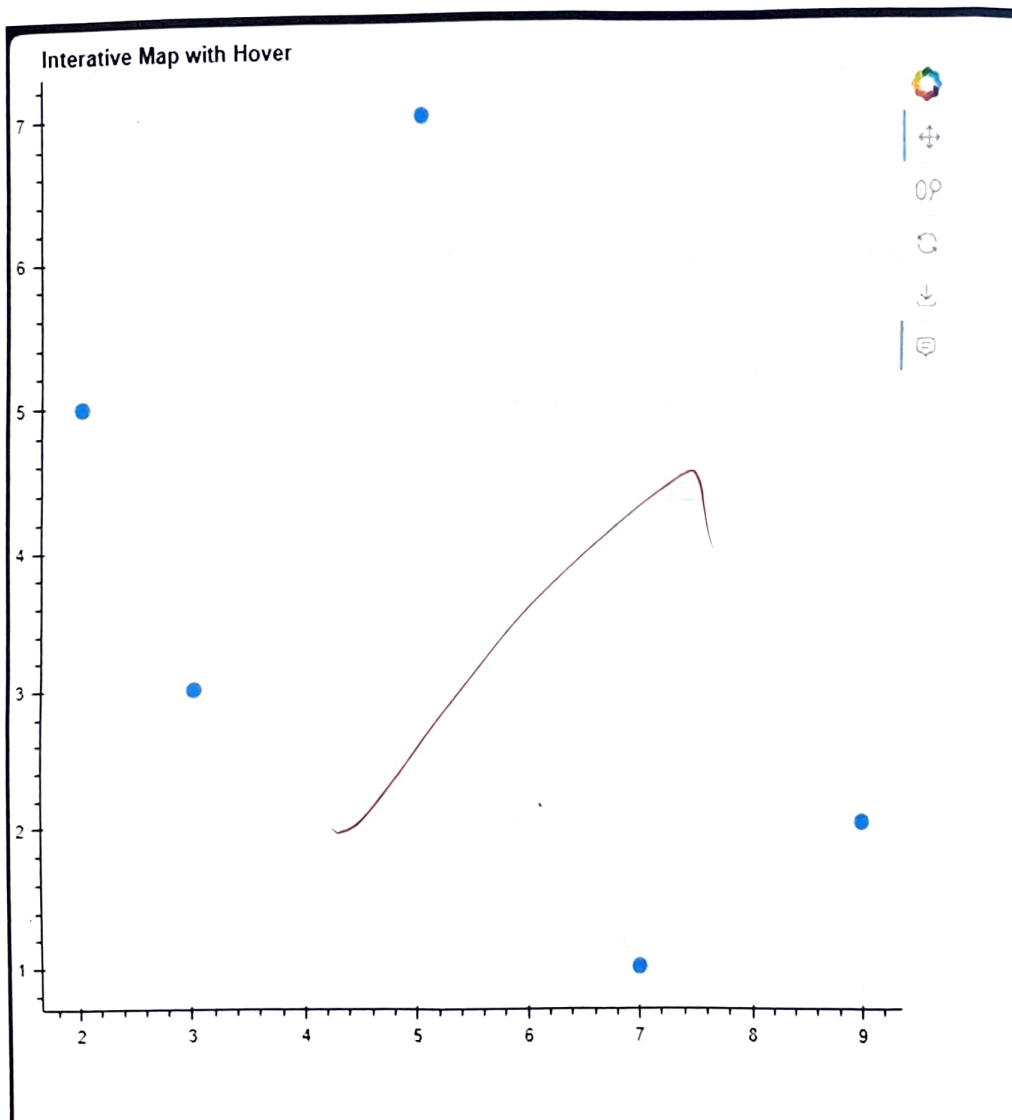


```
[13]: from bokeh.io import output_file, show
      from bokeh.plotting import figure
      from bokeh.models import ColumnDataSource, HoverTool

[14]: data={'x':[9,3,5,7,2], 'y':[2,3,7,1,5], 'desc':['G','S','O','H','W']}

[15]: source=ColumnDataSource(data=data)

[16]: p=figure(title="Interactive Map with Hover", tools="pan,wheel_zoom,reset,save")
      p.scatter(data['x'], data['y'], marker='o', size=10)
      hover=HoverTool()
      hover.tooltips=[("Description", "@desc"), ("x,y", "(x,y)", "$x,$y")]
      p.add_tools(hover)
      output_file("Interactive_map.html")
      show(p)
```



RESULT

Thus the data analysis and representation on a map using various map datasets with mouse rollover effect interaction has been completed successfully

EX: No: 07

BUILDING CARTOGRAPHIC VISUALIZATION

DATE: 28/08/25

AIM:

Build cartographic visualization for multiple datasets involving various countries of the world, states and districts in India etc...

PROCEDURE :**STEP-1: COLLECT DATASETS:**

Gather the datasets containing geographical information for countries, states or districts. Make sure these datasets include the necessary attributes for mapping (eg: country/state/district names, codes and relevant data).

STEP-2: INSTALL REQUIRED LIBRARIES:

`pip install geopandas matplotlib`

STEP-3: LOAD GEOGRAPHIC DATA:

Use `geopandas` to load the geographic data for countries, states or districts. Make sure to match the geographical data with your datasets based on the common attributes.

STEP-4: MERGE DATASETS:

Merge your datasets with the geographic data based on common attributes. This step is crucial for linking your data to the corresponding geographic regions.

STEP-5: CREATE CARTOGRAPHIC VISUALIZATIONS:

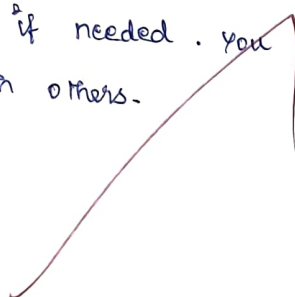
Use `matplotlib` to create cartographic visualizations. You can create separate plots for different datasets or overlay them on a single map.

STEP-6: CUSTOMIZE AND ENHANCE:

Customize your visualizations based on your needs. You can add legends, labels, title and other elements to enhance the interpretability of your maps.

STEP-7: SAVE AND SHARE :

Save your visualizations as image files or interactive plots if needed. You can then share these visualizations with others.



```
In [1]: 1 pip install geopandas bokeh pandas shapely
```

```
Defaulting to user installation because normal site-packages is not writeable
Requirement already satisfied: geopandas in c:\users\srmvel\appdata\roaming\python\python311\site-packages (1.0.1)
Requirement already satisfied: bokeh in c:\programdata\anaconda3\lib\site-packages (3.2.1)
Requirement already satisfied: pandas in c:\programdata\anaconda3\lib\site-packages (2.0.3)
Requirement already satisfied: shapely in c:\users\srmvel\appdata\roaming\python\python311\site-packages (2.0.6)
Requirement already satisfied: numpy>=1.22 in c:\programdata\anaconda3\lib\site-packages (from geopandas) (1.24.3)
Requirement already satisfied: pyogrio>=0.7.2 in c:\users\srmvel\appdata\roaming\python\python311\site-packages (from geopandas) (0.10.0)
Requirement already satisfied: packaging in c:\programdata\anaconda3\lib\site-packages (from geopandas) (23.1)
Requirement already satisfied: pyproj>=3.3.0 in c:\users\srmvel\appdata\roaming\python\python311\site-packages (from geopandas) (3.7.0)
Requirement already satisfied: Jinja2>=2.9 in c:\programdata\anaconda3\lib\site-packages (from bokeh) (3.1.2)
Requirement already satisfied: contourpy>=1 in c:\programdata\anaconda3\lib\site-packages (from bokeh) (1.0.5)
Requirement already satisfied: pillow>=7.1.0 in c:\programdata\anaconda3\lib\site-packages (from bokeh) (9.4.0)
Requirement already satisfied: PyYAML>=3.10 in c:\programdata\anaconda3\lib\site-packages (from bokeh) (6.0)
Requirement already satisfied: tornado>=5.1 in c:\programdata\anaconda3\lib\site-packages (from bokeh) (6.3.2)
Requirement already satisfied: xyzservices>=2021.09.1 in c:\programdata\anaconda3\lib\site-packages (from bokeh) (2022.9.0)
Requirement already satisfied: python-dateutil>=2.8.2 in c:\programdata\anaconda3\lib\site-packages (from pandas) (2.8.2)
Requirement already satisfied: pytz>=2020.1 in c:\programdata\anaconda3\lib\site-packages (from pandas) (2023.3.post1)
Requirement already satisfied: tzdata>=2022.1 in c:\programdata\anaconda3\lib\site-packages (from pandas) (2023.3)
Requirement already satisfied: MarkupSafe>=2.0 in c:\programdata\anaconda3\lib\site-packages (from Jinja2>=2.9->bokeh) (2.1.1)
Requirement already satisfied: certifi in c:\programdata\anaconda3\lib\site-packages (from pyogrio>=0.7.2->geopandas) (2023.7.2)
Requirement already satisfied: six>=1.5 in c:\programdata\anaconda3\lib\site-packages (from python-dateutil>=2.8.2->pandas) (1.16.0)
Note: you may need to restart the kernel to use updated packages.
```

```
In [8]: 1 from bokeh.io import output_file, show
2 from bokeh.plotting import figure
3 from bokeh.models import GeoJSONDataSource, HoverTool
4 import geopandas as gpd
```

```
In [9]: 1 world = gpd.read_file("C:/Users/srmvel/Downloads/ne_110m_admin_0_countries (1)/ne_110m_admin_0_countries.shp")
```

```
In [10]: 1 geo_source = GeoJSONDataSource(geopandas.world.to_json())
```

```
In [11]: 1 p = figure(title="Interactive Map with Hover", tools="pan,wheel_zoom,reset,save", x_axis_location=None, y_axis_location=None
2 p.grid.grid_line_color = "grey"
```

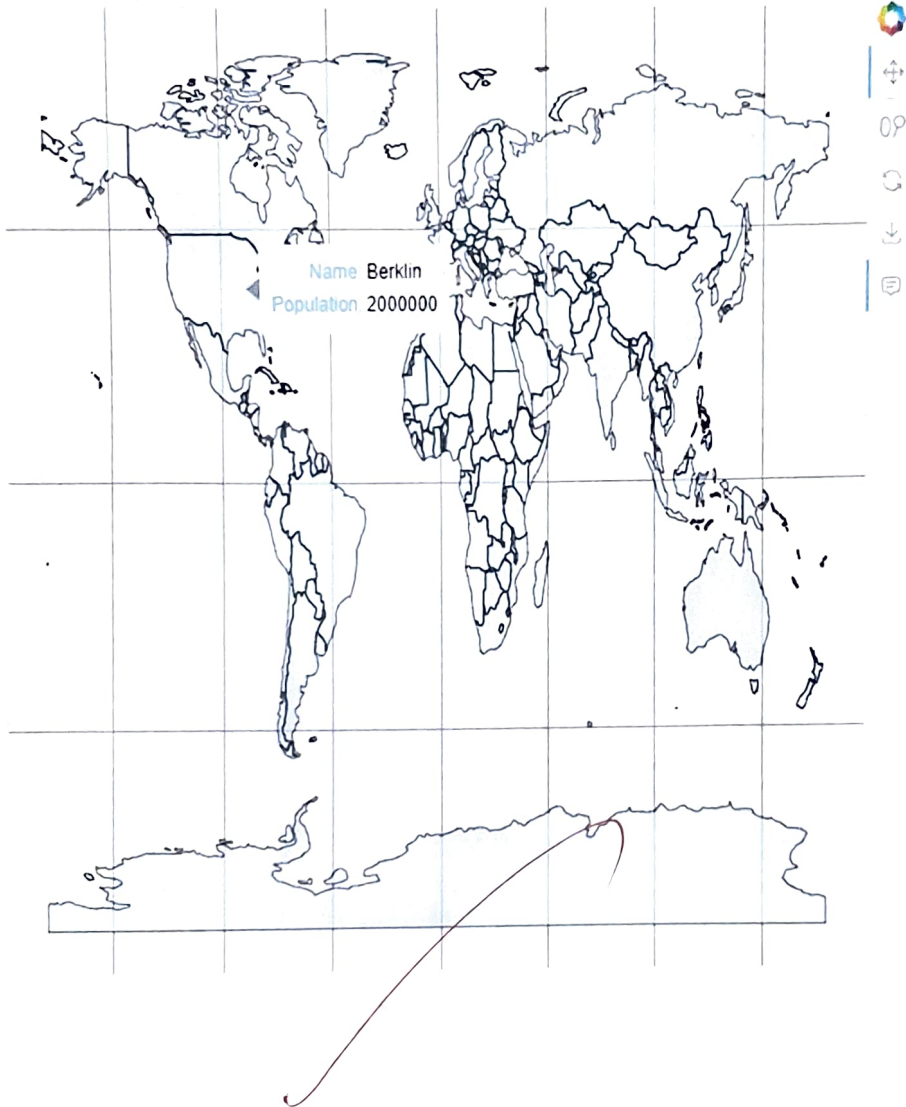
```
In [12]: 1 p.patches('xs', 'ys', source=geo_source, fill_color='lightblue', line_color='black', line_width=0.7, fill_alpha=0.5)
```

```
Out[12]: GlyphRenderer(id='p1077', ...)
```

```
In [13]: 1 hover = HoverTool()
2 hover.tooltips = [ ("Name", "Berlin"), ("Population", "2000000") ]
3 p.add_tools(hover)
```

```
In [14]: 1 output_file("interactive_map.html")
2 show(p)
```

Interactive Map with Hover



RESULT:

As per Build cartographic visualization for multiple datasets involving various countries of the world; has been visualization successfully.

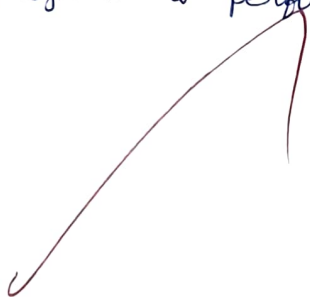
EX: NO : 08

DATE : 04/09/25

PERFORM EDA ON WINE QUALITY
DATASET

AIM :

TO write a program to perform EDA on wine
quality dataset.



```
In [63]: 1 import pandas as pd
2 import numpy as np
3 import matplotlib.pyplot as plt
4 import seaborn as sns
```

```
In [64]: 1 data = pd.read_csv("C:/Users/srmve/Downloads/WineQT.csv")
2 data
```

```
Out[64]:
```

	fixed acidity	volatile acidity	citric acid	residual sugar	chlorides	free sulfur dioxide	total sulfur dioxide	density	pH	sulphates	alcohol	quality	id
0	7.4	0.700	0.00	1.9	0.076	11.0	34.0	0.99780	3.51	0.56	9.4	5	0
1	7.8	0.880	0.00	2.6	0.098	25.0	67.0	0.99680	3.20	0.68	9.8	5	1
2	7.8	0.760	0.04	2.3	0.092	15.0	54.0	0.99700	3.26	0.65	9.8	5	2
3	11.2	0.280	0.56	1.9	0.075	17.0	60.0	0.99800	3.16	0.58	9.8	6	3
4	7.4	0.700	0.00	1.9	0.076	11.0	34.0	0.99780	3.51	0.56	9.4	5	4
...	...	...	...	...	...	...	...	...	...	...	...	...	...
1138	6.3	0.510	0.13	2.3	0.076	29.0	40.0	0.99574	3.42	0.75	11.0	6	1592
1139	6.8	0.620	0.08	1.9	0.068	28.0	38.0	0.99651	3.42	0.82	9.5	6	1593
1140	6.2	0.600	0.08	2.0	0.090	32.0	44.0	0.99490	3.45	0.58	10.5	5	1594
1141	5.9	0.550	0.10	2.2	0.062	39.0	51.0	0.99512	3.52	0.76	11.2	6	1595
1142	5.9	0.645	0.12	2.0	0.075	32.0	44.0	0.99547	3.57	0.71	10.2	5	1597

1143 rows x 13 columns

```
In [65]: 1 data.head()
```

```
Out[65]:
```

	fixed acidity	volatile acidity	citric acid	residual sugar	chlorides	free sulfur dioxide	total sulfur dioxide	density	pH	sulphates	alcohol	quality	id
0	7.4	0.70	0.00	1.9	0.076	11.0	34.0	0.9978	3.51	0.56	9.4	5	0
1	7.8	0.88	0.00	2.6	0.098	25.0	67.0	0.9968	3.20	0.68	9.8	5	1
2	7.8	0.76	0.04	2.3	0.092	15.0	54.0	0.9970	3.26	0.65	9.8	5	2
3	11.2	0.28	0.56	1.9	0.075	17.0	60.0	0.9980	3.16	0.58	9.8	6	3
4	7.4	0.70	0.00	1.9	0.076	11.0	34.0	0.9978	3.51	0.56	9.4	5	4

```
In [66]: 1 data.info()
```

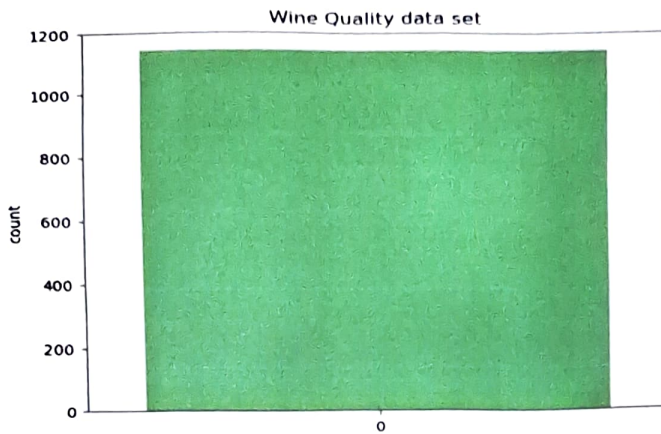
```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1143 entries, 0 to 1142
Data columns (total 13 columns):
 #   Column              Non-Null Count  Dtype
---  -
 0   fixed acidity        1143 non-null   float64
 1   volatile acidity     1143 non-null   float64
 2   citric acid          1143 non-null   float64
 3   residual sugar       1143 non-null   float64
 4   chlorides            1143 non-null   float64
 5   free sulfur dioxide  1143 non-null   float64
 6   total sulfur dioxide 1143 non-null   float64
 7   density              1143 non-null   float64
 8   pH                  1143 non-null   float64
 9   sulphates           1143 non-null   float64
10   alcohol             1143 non-null   float64
11   quality              1143 non-null   int64
12   id                  1143 non-null   int64
dtypes: float64(11), int64(2)
memory usage: 116.2 KB
```

```
In [67]: 1 data.describe()
```

```
Out[67]:
```

	fixed acidity	volatile acidity	citric acid	residual sugar	chlorides	free sulfur dioxide	total sulfur dioxide	density	pH	sulphates	alcohol	qu
count	1143.000000	1143.000000	1143.000000	1143.000000	1143.000000	1143.000000	1143.000000	1143.000000	1143.000000	1143.000000	1143.000000	1143.000000
mean	8.311111	0.531339	0.268364	2.532152	0.086933	15.615486	45.914698	0.996730	3.311015	0.657708	10.442111	5.651
std	1.747595	0.179633	0.196686	1.355917	0.047267	10.250486	32.782130	0.001925	0.156664	0.170399	1.082196	0.801
min	4.600000	0.120000	0.000000	0.900000	0.012000	1.000000	6.000000	0.990070	2.740000	0.330000	8.400000	3.000
25%	7.100000	0.392500	0.090000	1.900000	0.070000	7.000000	21.000000	0.995570	3.205000	0.550000	9.500000	5.000
50%	7.900000	0.520000	0.250000	2.200000	0.079000	13.000000	37.000000	0.996680	3.310000	0.620000	10.200000	6.000
75%	9.100000	0.640000	0.420000	2.600000	0.090000	21.000000	61.000000	0.997845	3.400000	0.730000	11.100000	6.000
max	15.900000	1.580000	1.000000	15.500000	0.611000	68.000000	289.000000	1.003660	4.010000	2.000000	14.900000	8.000

```
In [68]: 1 sns.countplot(data['quality'], color='green', alpha=0.7)
2 plt.title("Wine Quality data set")
3 plt.show()
```

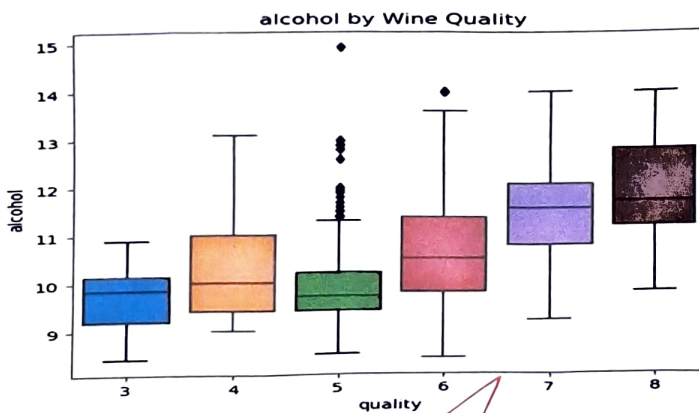



```
In [69]: 1 features = ['residual sugar', 'quality', 'density', 'alcohol']
2 for feature in features:
3     plt.figure(figsize=(6,5))
4     sns.boxplot(x='quality', y=feature, data=data)
5     plt.title(f'{feature} by Wine Quality')
6     plt.show()
```

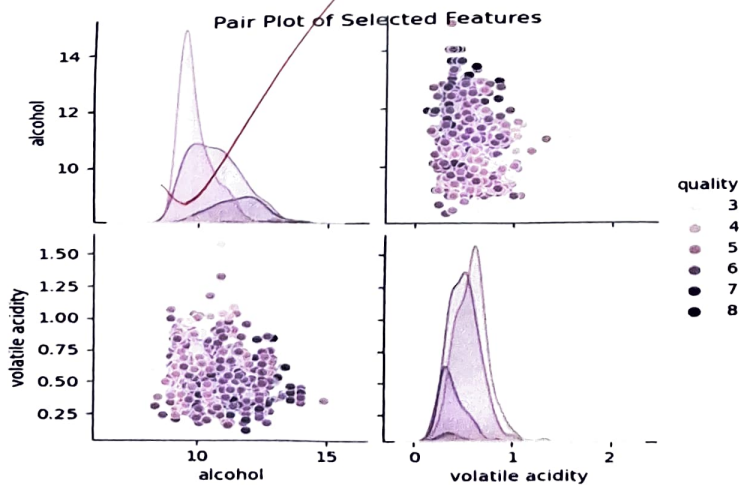
<Figure size 600x500 with 0 Axes>

<Figure size 600x500 with 0 Axes>

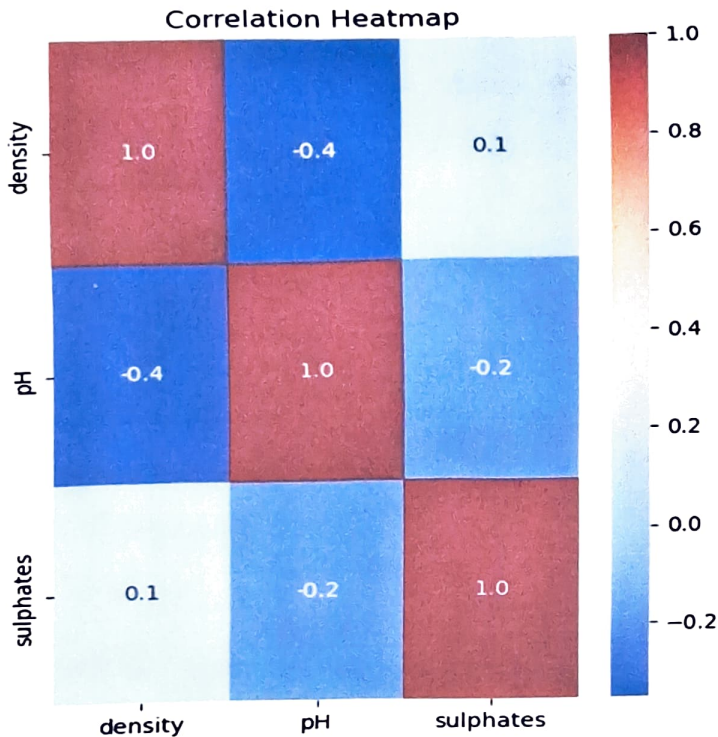
<Figure size 600x500 with 0 Axes>



```
In [70]: 1 sns.pairplot(data, vars=['alcohol', 'volatile acidity'], hue='quality', diag_kind='kde')
2 plt.suptitle("Pair Plot of Selected Features")
3 plt.show()
```



```
In [80]: 1 corr_matrix = data[['density', 'pH', 'sulphates']].corr ()
2 plt.figure (figsize = (5,6))
3 sns.heatmap (corr_matrix, annot=True, cmap="coolwarm", fmt=".1f")
4 plt.title ("Correlation Heatmap")
5 plt.show ()
```

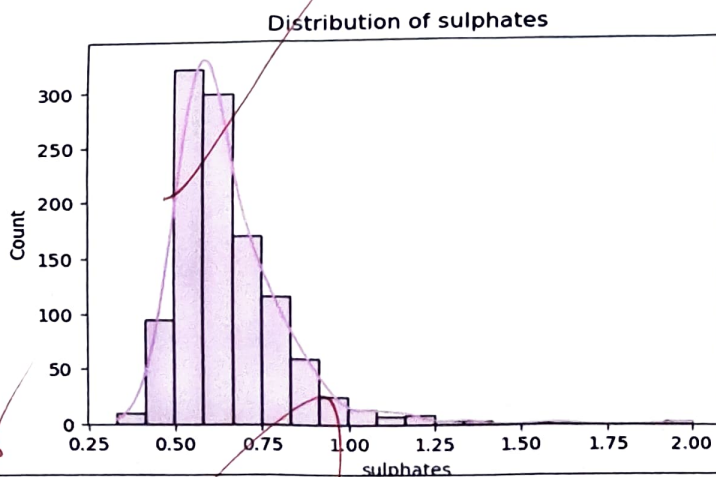


```
In [79]: 1 features = ['alcohol', 'density', 'citric acid', 'sulphates']
2 for feature in features:
3     plt.figure (figsize = (6, 4))
4     sns.histplot (data [feature], kde=True, bins=20,color='violet')
5     plt.title (f"Distribution of {feature}")
6     plt.show ()
```

<Figure size 600x400 with 0 Axes>

<Figure size 600x400 with 0 Axes>

<Figure size 600x400 with 0 Axes>



RESULT: Thus the above program to perform FDA on wine quality Data set.

EX: No: 09

VISUALIZING VARIOUS EDA TECHNIQUES

DATE: 25/09/25

AS CASE STUDY FOR IRIS DATASET

AIM:

The mini project to predict the time taken to solve a problem given the current status of the user using Random Forest Regression model.

PROCEDURE:

IMPORT LIBRARIES:

Start by importing the necessary libraries and loading the dataset.

DESCRIPTIVE STATISTICS:

compute and display descriptive statistics

CHECK FOR MISSING VALUES:

verify if there are any missing values in the dataset

VISUALIZE DATA DISTRIBUTION:

visualize the distribution of numerical variables.

CORRELATION HEATMAP:

Examine the correlation between numerical variables

BOXPLOTS FOR CATEGORICAL VARIABLES:

Use boxplots with kernel density estimation for better visualization.

CORRELATION BETWEEN FEATURES:

visualize pair-wise feature correlations.

```
In [17]: 1 import pandas as pd
2 import numpy as np
3 import matplotlib.pyplot as plt
4 import seaborn as sns
5 import warnings
6 warnings.filterwarnings("ignore")
```

```
In [18]: 1 data = pd.read_csv("C:/Users/srmve/Downloads/Iris (3).csv")
2 data
```

```
Out[18]:
```

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
	0	1	5.1	3.5	1.4	0.2 Iris-setosa
	1	2	4.9	3.0	1.4	0.2 Iris-setosa
	2	3	4.7	3.2	1.3	0.2 Iris-setosa
	3	4	4.6	3.1	1.5	0.2 Iris-setosa
	4	5	5.0	3.6	1.4	0.2 Iris-setosa
...	...	...	...	...	...	...
	145	146	6.7	3.0	5.2	2.3 Iris-virginica
	146	147	6.3	2.5	5.0	1.9 Iris-virginica
	147	148	6.5	3.0	5.2	2.0 Iris-virginica
	148	149	6.2	3.4	5.4	2.3 Iris-virginica
	149	150	5.9	3.0	5.1	1.8 Iris-virginica

150 rows x 6 columns

```
In [19]: 1 data.head()
```

```
Out[19]:
```

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
0	1	5.1	3.5	1.4	0.2	Iris-setosa
1	2	4.9	3.0	1.4	0.2	Iris-setosa
2	3	4.7	3.2	1.3	0.2	Iris-setosa
3	4	4.6	3.1	1.5	0.2	Iris-setosa
4	5	5.0	3.6	1.4	0.2	Iris-setosa

```
In [20]: 1 data.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 6 columns):
#   Column              Non-Null Count  Dtype  
---  --
0    Id                   150 non-null    int64   
1    SepalLengthCm        150 non-null    float64  
2    SepalWidthCm         150 non-null    float64  
3    PetalLengthCm        150 non-null    float64  
4    PetalWidthCm         150 non-null    float64  
5    Species              150 non-null    object  
dtypes: float64(4), int64(1), object(1)
memory usage: 7.2+ KB
```

```
In [21]: 1 data.describe()
```

```
Out[21]:
```

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm
count	150.000000	150.000000	150.000000	150.000000	150.000000
mean	75.500000	5.843333	3.054000	3.758667	1.198667
std	43.445368	0.828066	0.433594	1.764420	0.763161
min	1.000000	4.300000	2.000000	1.000000	0.100000
25%	38.250000	5.100000	2.800000	1.600000	0.300000
50%	75.500000	5.800000	3.000000	4.350000	1.300000
75%	112.750000	6.400000	3.300000	5.100000	1.800000
max	150.000000	7.900000	4.400000	6.900000	2.500000

```
In [22]: 1 data.isnull().sum()
```

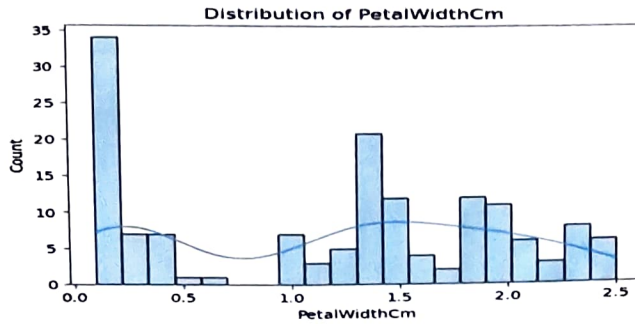
```
Out[22]: Id                0
SepalLengthCm            0
SepalWidthCm             0
PetalLengthCm            0
PetalWidthCm             0
Species                  0
dtype: int64
```

```
In [23]: 1 features = ['SepalLengthCm', 'SepalWidthCm', 'PetalLengthCm', 'PetalWidthCm']
2 for feature in features:
3     plt.figure(figsize=(6, 4))
4     sns.histplot(data[data[feature]], kde=True, bins=20)
5     plt.title(f'Distribution of {feature}')
6     plt.show()
```

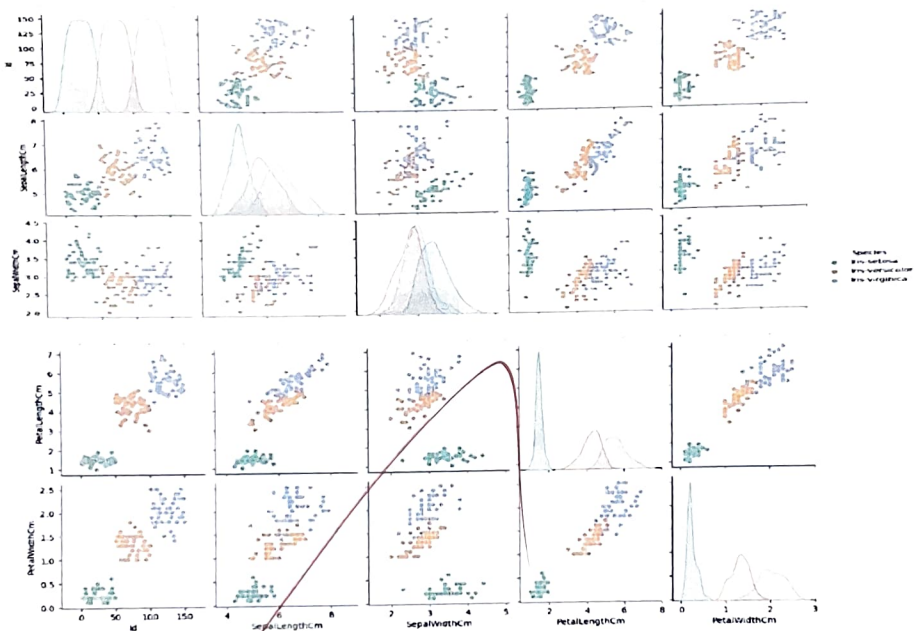

<Figure size 600x400 with 0 Axes>

<Figure size 600x400 with 0 Axes>

<Figure size 600x400 with 0 Axes>



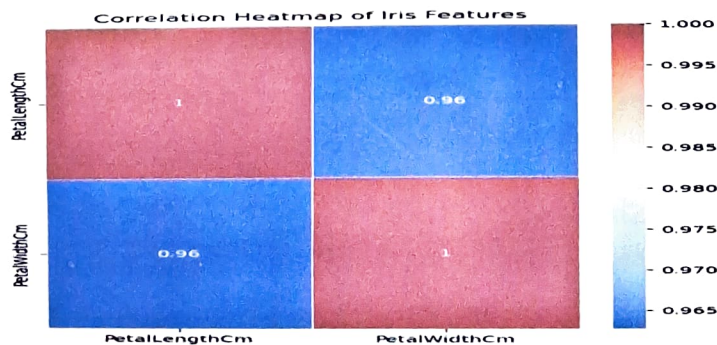
```
In [16]: 1 sns.pairplot(data, hue="Species", palette="Set2", diag_kind="kde")
        2 plt.show()
```



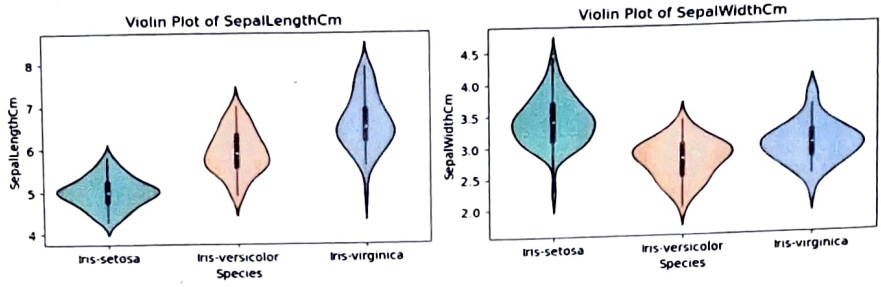
```
In [24]: 1 correlation_matrix = data[['PetalLengthCm', 'PetalWidthCm']].corr()
        2 print(correlation_matrix)
```

```
PetalLengthCm  PetalLengthCm  PetalWidthCm
PetalLengthCm  1.000000      0.962757
PetalWidthCm   0.962757      1.000000
```

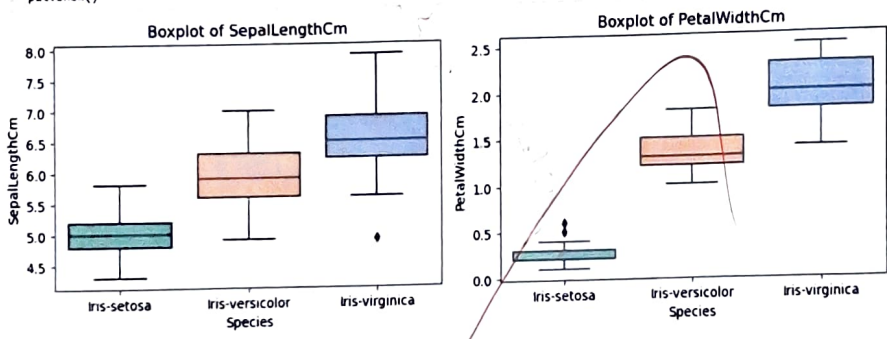
```
In [26]: 1 plt.figure(figsize=(6,6))
        2 sns.heatmap(correlation_matrix, annot=True, cmap="coolwarm", linewidths=0.5)
        3 plt.title('Correlation Heatmap of Iris Features')
        4 plt.show()
```




```
In [52]: 1 plt.figure(figsize=(10, 6))
2
3 for i, feature in enumerate(['SepalLengthCm', 'SepalWidthCm']):
4     plt.subplot(2, 2, i + 1)
5     sns.violinplot(x='Species', y=feature, data=data, palette='Set2')
6     plt.title(f'Violin Plot of {feature}')
7     plt.tight_layout()
8
9 plt.show()
```



```
In [53]: 1 plt.figure(figsize=(10, 6))
2
3 for i, feature in enumerate(['SepalLengthCm', 'PetalWidthCm']):
4     plt.subplot(2, 2, i + 1)
5     sns.boxplot(x='Species', y=feature, data=data, palette='Set2')
6     plt.title(f'Boxplot of {feature}')
7     plt.tight_layout()
8
9 plt.show()
```



RESULT: This case study provides a comprehensive analysis of the Iris datasets, including data exploration, descriptive statistics and visualization of data distributions, correlations, correlation analysis and feature-specific visualizations.

DATA EXPLORATION AND VISUALIZATION

WORKING WITH PANDAS DATAFRAMES (PROJECT)

```
In [37]: 1 import numpy as np
        2 import pandas as pd
        3 import matplotlib.pyplot as plt
        4 import seaborn as sns
```

```
In [38]: 1 df=pd.read_csv("C:/Users/SRMVEG/Downloads/ecom.data.csv")
        2 df
```

```
Out[38]:
```

	ProductID	ProductName	Category	Price	QuantitySold	Promotion	Discount	Rating	CustomerAge	CustomerGender	CustomerLocation	PurchaseDate
0	1001	Badminton Set	Electronics	50.0	77	Yes	32	4.0	64	Female	Australia	2022-01-01
1	1002	Curtains	Electronics	196.0	38	Yes	45	NaN	24	Male	China	2022-01-02
2	1003	Hoodie	Beauty	336.0	8	No	4	NaN	36	Male	India	2022-01-03
3	1004	Mascara	Beauty	191.0	78	No	42	1.0	39	Male	Australia	2022-01-04
4	1005	Gaming Console	Sports	102.0	74	No	47	NaN	52	Female	Russia	2022-01-05
5	1006	Sofa	Home & Garden	353.0	1	No	33	5.0	47	Male	Brazil	2022-01-06
6	1007	Badminton Set	Electronics	118.0	50	No	39	1.0	46	Male	Russia	2022-01-07
7	1008	Jacket	Home & Garden	410.0	30	Yes	10	1.0	39	Female	Germany	2022-01-08

```
In [39]: 1 df.head()
```

```
Out[39]:
```

	ProductID	ProductName	Category	Price	QuantitySold	Promotion	Discount	Rating	CustomerAge	CustomerGender	CustomerLocation	PurchaseDate
0	1001	Badminton Set	Electronics	50.0	77	Yes	32	4.0	64	Female	Australia	2022-01-01
1	1002	Curtains	Electronics	196.0	38	Yes	45	NaN	24	Male	China	2022-01-02
2	1003	Hoodie	Beauty	336.0	8	No	4	NaN	36	Male	India	2022-01-03
3	1004	Mascara	Beauty	191.0	78	No	42	1.0	39	Male	Australia	2022-01-04
4	1005	Gaming Console	Sports	102.0	74	No	47	NaN	52	Female	Russia	2022-01-05

```
In [40]: 1 df.tail()
```

```
Out[40]:
```

	ProductID	ProductName	Category	Price	QuantitySold	Promotion	Discount	Rating	CustomerAge	CustomerGender	CustomerLocation	PurchaseDate
995	1996	Coffee Maker	Sports	118.0	17	Yes	34	NaN	36	Female	Brazil	2024-09-22
996	1997	Exercise Bike	Home & Garden	180.0	0	Yes	21	1.0	38	Female	France	2024-09-23
997	1998	Blush	Electronics	136.0	54	No	39	NaN	41	Male	Spain	2024-09-24
998	1999	Camera	Sports	267.0	96	No	22	NaN	55	Female	France	2024-09-25
999	2000	Tennis Racket	Electronics	470.0	8	Yes	2	2.0	21	Male	India	2024-09-26

```
In [41]: 1 df.isnull().sum()
```

```
Out[42]: ProductID      0
ProductName      0
Category          0
Price            980
QuantitySold      0
Promotion         0
Discount          0
Rating           133
CustomerAge       0
CustomerGender    0
CustomerLocation  0
PurchaseDate      0
dtype: int64
```

```
In [42]: df.isnull().sum().sum()
```

```
Out[42]: 153
```

```
In [43]: df.count()
```

```
Out[43]: ProductID      1000
ProductName      1000
Category          1000
Price            980
QuantitySold      1000
Promotion         1000
Discount          1000
Rating           867
CustomerAge       1000
CustomerGender    1000
CustomerLocation  1000
PurchaseDate      1000
dtype: int64
```

```
In [44]: df.info()
```

```
<class pandas.core.frame.DataFrame>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 12 columns):
ProductID      1000 non-null int64
ProductName     1000 non-null object
Category        1000 non-null object
Price          980 non-null float64
QuantitySold    1000 non-null int64
Promotion       1000 non-null object
Discount        1000 non-null int64
Rating         867 non-null float64
CustomerAge     1000 non-null int64
CustomerGender  1000 non-null object
CustomerLocation 1000 non-null object
PurchaseDate    1000 non-null object
dtypes: float64(2), int64(4), object(6)
memory usage: 93.8+ KB
```

```
In [45]: df.describe()
```

```
Out[45]:
```

	ProductID	Price	QuantitySold	Discount	Rating	CustomerAge
count	1000.000000	980.000000	1000.000000	1000.000000	867.000000	1000.000000
mean	1500.500000	272.608163	43.293000	24.966000	3.377163	40.347000
std	288.819436	129.733952	31.107183	14.346521	1.296499	13.269237
min	1001.000000	50.000000	-10.000000	0.000000	1.000000	18.000000
25%	1250.750000	165.000000	17.000000	13.000000	3.000000	29.000000
50%	1500.500000	269.000000	41.000000	25.000000	4.000000	40.000000
75%	1750.250000	385.250000	70.000000	37.250000	4.000000	52.000000
max	2900.000000	499.000000	99.000000	49.000000	5.000000	64.000000

```
In [46]: df.fillna('*****')
```

```
Out[46]:
```

	ProductID	ProductName	Category	Price	QuantitySold	Promotion	Discount	Rating	CustomerAge	CustomerGender	CustomerLocation	PurchaseDate
0	1001	Badminton Set	Electronics	50	77	Yes	32	4	64	Female	Australia	2022-01-01
1	1002	Curtains	Electronics	196	38	Yes	45	*****	24	Male	China	2022-01-02
2	1003	Hoodie	Beauty	336	8	No	4	*****	36	Male	India	2022-01-03
3	1004	Mascara	Beauty	191	78	No	42	1	39	Male	Australia	2022-01-04
4	1005	Gaming Console	Sports	102	74	No	47	*****	52	Female	Russia	2022-01-05
5	1006	Sofa	Home & Garden	353	1	No	33	5	47	Male	Brazil	2022-01-06
6	1007	Badminton Set	Electronics	118	50	No	39	1	46	Male	Russia	2022-01-07
7	1008	Jacket	Home & Garden	410	30	Yes	10	1	39	Female	Germany	2022-01-08

```
In [47]: df.bfill()
```

```
Out[47]:
```

	ProductID	ProductName	Category	Price	QuantitySold	Promotion	Discount	Rating	CustomerAge	CustomerGender	CustomerLocation	PurchaseDate
0	1001	Badminton Set	Electronics	50.0	77	Yes	32	4.0	64	Female	Australia	2022-01-01
1	1002	Curtains	Electronics	196.0	38	Yes	45	1.0	24	Male	China	2022-01-02
2	1003	Hoodie	Beauty	336.0	8	No	4	1.0	36	Male	India	2022-01-03
3	1004	Mascara	Beauty	191.0	78	No	42	1.0	39	Male	Australia	2022-01-04
4	1005	Gaming Console	Sports	102.0	74	No	47	5.0	52	Female	Russia	2022-01-05
5	1006	Sofa	Home & Garden	353.0	1	No	33	5.0	47	Male	Brazil	2022-01-06
6	1007	Badminton Set	Electronics	118.0	50	No	39	1.0	46	Male	Russia	2022-01-07
7	1008	Jacket	Home & Garden	410.0	30	Yes	10	1.0	39	Female	Germany	2022-01-08

```
In [48]: df.ffill()
```

```
Out[48]:
```

	ProductID	ProductName	Category	Price	QuantitySold	Promotion	Discount	Rating	CustomerAge	CustomerGender	CustomerLocation	PurchaseDate
0	1001	Badminton Set	Electronics	50.0	77	Yes	32	4.0	64	Female	Australia	2022-01-01
1	1002	Curtains	Electronics	196.0	38	Yes	45	4.0	24	Male	China	2022-01-02
2	1003	Hoodie	Beauty	336.0	8	No	4	4.0	36	Male	India	2022-01-03
3	1004	Mascara	Beauty	191.0	78	No	42	1.0	39	Male	Australia	2022-01-04
4	1005	Gaming Console	Sports	102.0	74	No	47	1.0	52	Female	Russia	2022-01-05
5	1006	Sofa	Home & Garden	353.0	1	No	33	5.0	47	Male	Brazil	2022-01-06
6	1007	Badminton Set	Electronics	118.0	50	No	39	1.0	46	Male	Russia	2022-01-07
7	1008	Jacket	Home & Garden	410.0	30	Yes	10	1.0	39	Female	Germany	2022-01-08

```
In [49]: df["Price"]
```

```
Out[54]:
```

0	50.0
1	196.0
2	336.0
3	191.0
4	102.0
5	353.0
6	118.0
7	410.0
8	139.0
9	495.0
10	269.0
11	499.0
12	241.0
13	112.0
14	122.0
15	210.0
16	218.0
17	138.0
18	225.0
19	140.0

```
In [56]: 1 df.drop("Promotion",axis=1)
```

```
Out[56]:
```

	ProductID	ProductName	Category	Price	QuantitySold	Discount	Rating	CustomerAge	CustomerGender	CustomerLocation	PurchaseDate
0	1001	Badminton Set	Electronics	50.0	77	32	4.0	64	Female	Australia	2022-01-01
1	1002	Curtains	Electronics	196.0	38	45	NaN	24	Male	China	2022-01-02
2	1003	Hoodie	Beauty	336.0	8	4	NaN	36	Male	India	2022-01-03
3	1004	Mascara	Beauty	191.0	78	42	1.0	39	Male	Australia	2022-01-04
4	1005	Gaming Console	Sports	102.0	74	47	NaN	52	Female	Russia	2022-01-05
5	1006	Sofa	Home & Garden	353.0	1	33	5.0	47	Male	Brazil	2022-01-06
6	1007	Badminton Set	Electronics	118.0	50	39	1.0	46	Male	Russia	2022-01-07
7	1008	Jacket	Home & Garden	410.0	30	10	1.0	39	Female	Germany	2022-01-08
8	1009	Dining Table	Home & Garden	139.0	22	49	3.0	28	Female	France	2022-01-09
9	1010	T-Shirt	Beauty	495.0	39	2	1.0	37	Female	South Africa	2022-01-10
10	1011	T-Shirt	Sports	269.0	57	32	4.0	46	Male	Canada	2022-01-11

```
In [57]: 1 df.isnull()
```

```
Out[57]:
```

	ProductID	ProductName	Category	Price	QuantitySold	Promotion	Discount	Rating	CustomerAge	CustomerGender	CustomerLocation	PurchaseDate
0	False	False	False	False	False	False	False	False	False	False	False	False
1	False	False	False	False	False	False	False	True	False	False	False	False
2	False	False	False	False	False	False	False	True	False	False	False	False
3	False	False	False	False	False	False	False	False	False	False	False	False
4	False	False	False	False	False	False	False	True	False	False	False	False
5	False	False	False	False	False	False	False	False	False	False	False	False
6	False	False	False	False	False	False	False	False	False	False	False	False
7	False	False	False	False	False	False	False	False	False	False	False	False
8	False	False	False	False	False	False	False	False	False	False	False	False
9	False	False	False	False	False	False	False	False	False	False	False	False
10	False	False	False	False	False	False	False	False	False	False	False	False

```
In [58]: 1 pd.melt(df,id_vars=["Promotion"],var_name="new position",value_name="values")
```


Out[60]:

	Promotion	new position	values
0	Yes	ProductID	1001
1	Yes	ProductID	1002
2	No	ProductID	1003
3	No	ProductID	1004
4	No	ProductID	1005
5	No	ProductID	1006
6	No	ProductID	1007
7	Yes	ProductID	1008
8	Yes	ProductID	1009
9	Yes	ProductID	1010
10	Yes	ProductID	1011

In [61]: 1 df.stack()

```
Out[61]: 0 ProductID      1001
          ProductName  Badminton Set
          Category      Electronics
          Price         50
          QuantitySold   77
          Promotion      Yes
          Discount       32
          Rating         4
          CustomerAge    64
          CustomerGender Female
          CustomerLocation Australia
          PurchaseDate   2022-01-01
          1 ProductID      1002
            ProductName    Curtains
            Category      Electronics
            Price         196
            QuantitySold   38
            Promotion      Yes
            Discount       45
            CustomerAge    34
```

In [62]: 1 df.pivot_table(index='Category',columns="Discount",values="Price",aggfunc="sum")

Out[62]:

Discount	0	1	2	3	4	5	6	7	8	9	...	40	41	42	43	44	45	46	47	48
Category																				
Beauty	1099.0	477.0	1663.0	1839.0	1235.0	248.0	532.0	951.0	1244.0	887.0	...	1278.0	1272.0	990.0	844.0	1192.0	382.0	1166.0	1183.0	11
Clothing	1113.0	723.0	773.0	362.0	454.0	1018.0	1101.0	947.0	650.0	523.0	...	491.0	1840.0	2308.0	951.0	1200.0	776.0	428.0	1312.0	2
Electronics	856.0	1234.0	1345.0	407.0	1128.0	1392.0	636.0	1169.0	1078.0	770.0	...	769.0	791.0	877.0	1047.0	1247.0	724.0	1756.0	950.0	1
Home & Garden	451.0	1240.0	1901.0	366.0	689.0	680.0	511.0	1293.0	635.0	154.0	...	NaN	712.0	509.0	726.0	1064.0	3125.0	1544.0	1215.0	!
Sports	1022.0	570.0	2013.0	623.0	1074.0	328.0	854.0	1155.0	1046.0	1564.0	...	269.0	763.0	2504.0	495.0	286.0	2256.0	293.0	2230.0	17

5 rows x 50 columns

```
In [63]: df.transpose()
```

```
Out[63]:
```

	0	1	2	3	4	5	6	7	8	9	...	990	991	992	993
ProductID	1001	1002	1003	1004	1005	1006	1007	1008	1009	1010	...	1991	1992	1993	1994
ProductName	Badminton Set	Curtains	Hoodie	Mascara	Gaming Console	Sofa	Badminton Set	Jacket	Dining Table	T-Shirt	...	Jacket	Foundation	Lipstick	Tennis Racket
Category	Electronics	Electronics	Beauty	Beauty	Sports	Home & Garden	Electronics	Home & Garden	Home & Garden	Beauty	...	Clothing	Clothing	Electronics	Clothing
Price	50	196	336	191	102	353	118	410	139	495	...	362	214	363	70
Quantity Sold	77	38	8	78	74	1	50	30	22	39	...	-5	58	33	53
Promotion	Yes	Yes	No	No	No	No	No	Yes	Yes	Yes	...	Yes	No	Yes	Yes
Discount	32	45	4	42	47	33	39	10	49	2	...	3	26	17	20
Rating	4	NaN	NaN	1	NaN	5	1	1	3	1	...	5	3	NaN	2
CustomerAge	64	24	36	39	52	47	46	39	28	37	...	49	27	26	44
CustomerGender	Female	Male	Male	Male	Female	Male	Male	Female	Female	Female	...	Female	Male	Male	Male
CustomerLocation	Australia	China	India	Australia	Russia	Brazil	Russia	Germany	France	South Africa	...	France	India	India	South Africa
PurchaseDate	2022-01-01	2022-01-02	2022-01-03	2022-01-04	2022-01-05	2022-01-06	2022-01-07	2022-01-08	2022-01-09	2022-01-10	...	2024-09-17	2024-09-18	2024-09-19	2024-09-20

12 rows x 1000 columns

```
In [83]: cross_tab=pd.crosstab(df["Category"],df["Price"])
cross_tab
```

```
Out[83]:
```

	Price	50.0	51.0	52.0	53.0	54.0	56.0	57.0	58.0	59.0	60.0	...	489.0	490.0	491.0	492.0	493.0	494.0	495.0	496.0	498.0	499.0
Category																						
Beauty	0	0	1	0	0	1	0	0	2	1	...	1	1	1	0	0	0	1	0	0	1	
Clothing	0	1	0	1	2	0	0	0	0	0	...	0	1	0	0	1	0	1	0	1	1	
Electronics	1	0	3	0	0	0	2	0	1	0	...	0	0	0	1	0	0	0	1	0	0	
Home & Garden	0	0	1	0	1	0	0	0	1	0	...	0	0	0	0	1	1	0	1	0	2	
Sports	0	0	0	0	0	2	0	1	0	1	...	0	0	1	0	2	0	0	0	2	1	

5 rows x 395 columns

```
In [84]: df.rename(columns={"Price": "Amount"})
```

Out[85]:

	ProductID	ProductName	Category	Price	QuantitySold	Promotion	Discount	Rating	CustomerAge	CustomerGender	CustomerLocation	PurchaseDate
0	1001	Badminton Set	Electronics	50.0	77	Yes	32	4.0	64	Female	Australia	2022-01-01
3	1004	Mascara	Beauty	191.0	78	No	42	1.0	39	Male	Australia	2022-01-04
5	1006	Sofa	Home & Garden	353.0	1	No	33	5.0	47	Male	Brazil	2022-01-06
6	1007	Badminton Set	Electronics	118.0	50	No	39	1.0	46	Male	Russia	2022-01-07
7	1008	Jacket	Home & Garden	410.0	30	Yes	10	1.0	39	Female	Germany	2022-01-08
...	...	...	...	...	...	...	...	...	...	...	...	...
991	1992	Foundation	Clothing	214.0	58	No	26	3.0	27	Male	India	2024-09-18
993	1994	Tennis Racket	Clothing	70.0	53	Yes	20	2.0	44	Male	South Africa	2024-09-20
994	1995	Camera	Electronics	131.0	18	Yes	27	1.0	30	Female	Italy	2024-09-21
996	1997	Exercise Bike	Home & Garden	180.0	0	Yes	21	1.0	38	Female	France	2024-09-23
999	2000	Tennis Racket	Electronics	470.0	8	Yes	2	2.0	21	Male	India	2024-09-26

849 rows x 12 columns

```
In [86]: df.replace(to_replace=np.nan,method='bfill')
```

Out[86]:

	ProductID	ProductName	Category	Price	QuantitySold	Promotion	Discount	Rating	CustomerAge	CustomerGender	CustomerLocation	PurchaseDate
0	1001	Badminton Set	Electronics	50.0	77	Yes	32	4.0	64	Female	Australia	2022-01-01
1	1002	Curtains	Electronics	196.0	38	Yes	45	1.0	24	Male	China	2022-01-02
2	1003	Hoodie	Beauty	336.0	8	No	4	1.0	36	Male	India	2022-01-03
3	1004	Mascara	Beauty	191.0	78	No	42	1.0	39	Male	Australia	2022-01-04
4	1005	Gaming Console	Sports	102.0	74	No	47	5.0	52	Female	Russia	2022-01-05
...	...	...	...	...	...	...	...	...	...	...	...	...
995	1996	Coffee Maker	Sports	118.0	17	Yes	34	1.0	36	Female	Brazil	2024-09-22
996	1997	Exercise Bike	Home & Garden	180.0	0	Yes	21	1.0	38	Female	France	2024-09-23
997	1998	Blush	Electronics	136.0	54	No	39	2.0	41	Male	Spain	2024-09-24
998	1999	Camera	Sports	267.0	96	No	22	2.0	55	Female	France	2024-09-25
999	2000	Tennis Racket	Electronics	470.0	8	Yes	2	2.0	21	Male	India	2024-09-26

1000 rows x 12 columns

```
In [87]: df.drop_duplicates(subset='Category',keep="first")
```

Out[87]:

	ProductID	ProductName	Category	Price	QuantitySold	Promotion	Discount	Rating	CustomerAge	CustomerGender	CustomerLocation	PurchaseDate
0	1001	Badminton Set	Electronics	50.0	77	Yes	32	4.0	64	Female	Australia	2022-01-01
2	1003	Hoodie	Beauty	336.0	8	No	4	NaN	36	Male	India	2022-01-03
4	1005	Gaming Console	Sports	102.0	74	No	47	NaN	52	Female	Russia	2022-01-05
5	1006	Sofa	Home & Garden	353.0	1	No	33	5.0	47	Male	Brazil	2022-01-06
11	1012	Refrigerator	Clothing	499.0	95	Yes	10	3.0	58	Male	South Africa	2022-01-12

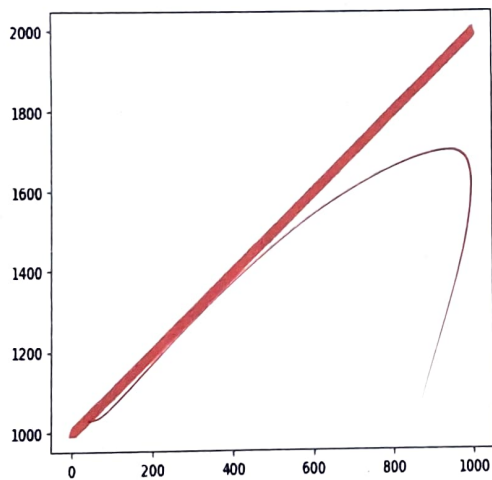
```
In [88]: df.duplicated(subset="Category",keep="first")
```

```
Out[88]: 0    False
          1     True
          2    False
          3     True
          4    False
          ...
          995   True
          996   True
          997   True
          998   True
          999   True
          Length: 1000, dtype: bool
```

```
In [89]: df["Rating"].mean()
```

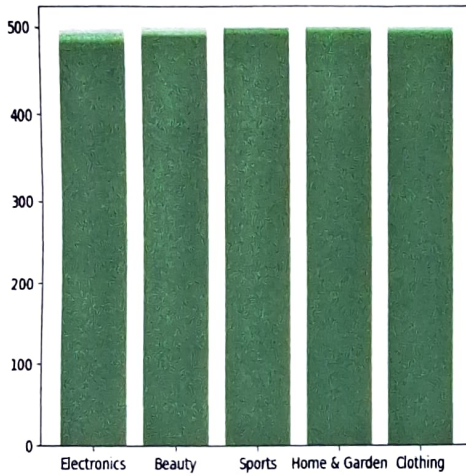
```
Out[89]: 3.3771626297577853
```

```
In [90]: plt.plot(df['ProductID'],marker='^',color='r',alpha=0.3)
          plt.show()
```



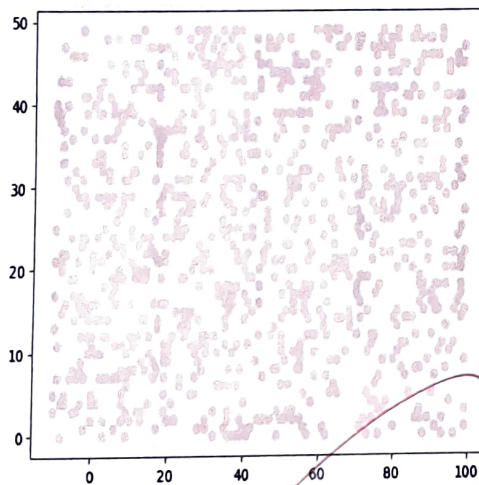
```
In [91]: plt.bar(df['Category'],df['Price'],color='g',alpha=0.3)
```

Out[91]: <BarContainer object of 1000 artists>



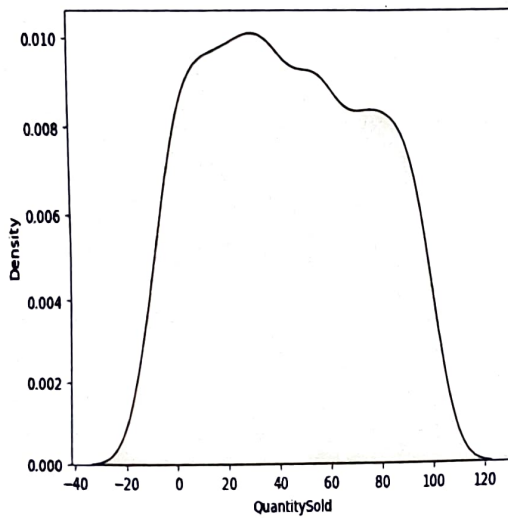
In [92]: `plt.scatter(df['QuantitySold'],df['Discount'],color='pink',marker='o')`

Out[92]: <matplotlib.collections.PathCollection at 0x1e54917a9d0>

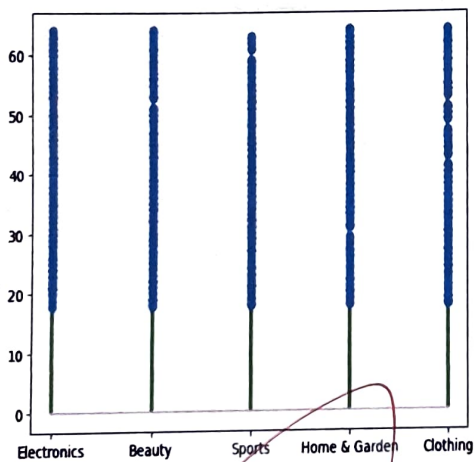


In [93]: `sns.kdeplot(df['QuantitySold'],fill=True,color="orange",edgecolor="black")`

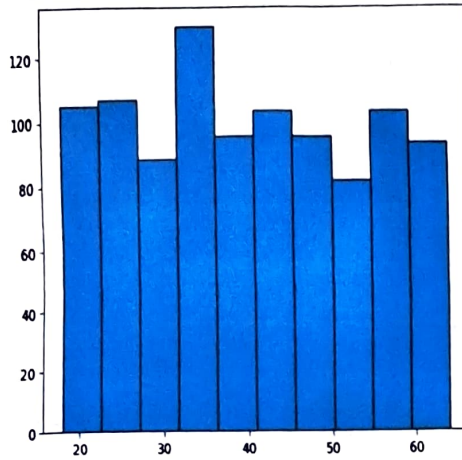
Out[93]: <Axes: xlabel='QuantitySold', ylabel='Density'>



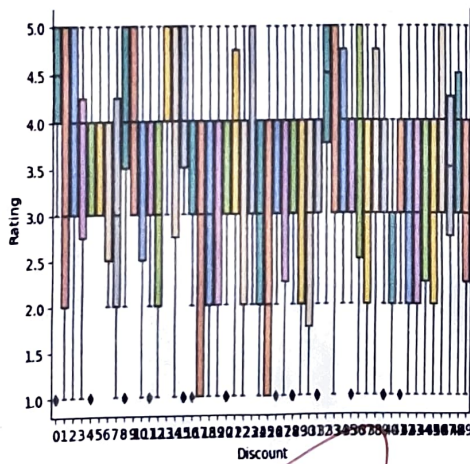
```
In [94]: plt.stem(df['Category'],df['CustomerAge'],basefat='violet',linefat='green',markerfat='bo')
Out[94]: <StemContainer object of 3 artists>
```



```
In [95]: plt.hist(df['CustomerAge'],edgecolor='black')
Out[95]: (array([105., 107., 88., 130., 95., 103., 95., 81., 103., 93.]),
array([18., 22.6, 27.2, 31.8, 36.4, 41., 45.6, 50.2, 54.8, 59.4, 64. ]),
<BarContainer object of 10 artists>)
```



```
In [104]: sns.boxplot(x=df['Discount'], y=df['Rating'], palette='Set2')
Out[104]: <Axes: xlabel='Discount', ylabel='Rating'>
```



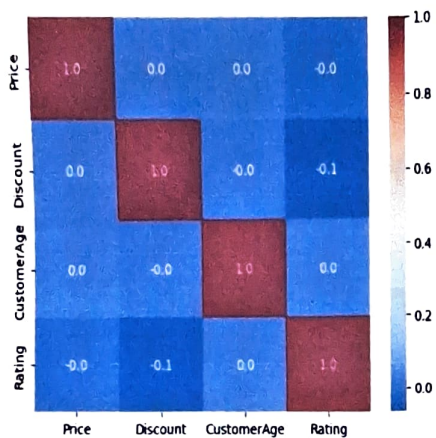
```
In [111]: corrmatrix=df[['Price','Discount','CustomerAge','Rating']].corr()
corrmatrix
```

```
Out[111]:
```

	Price	Discount	CustomerAge	Rating
Price	1.000000	0.003281	0.011532	-0.025802
Discount	0.003281	1.000000	-0.003740	-0.068589
CustomerAge	0.011532	-0.003740	1.000000	0.011061
Rating	-0.025802	-0.068589	0.011061	1.000000

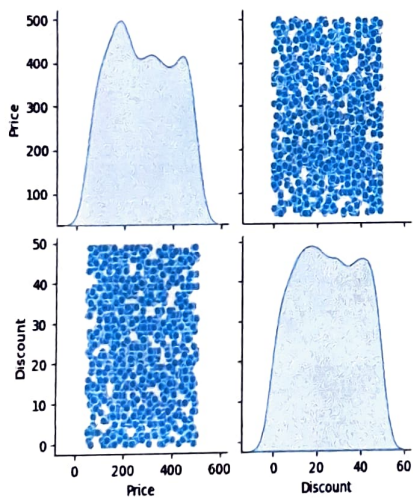
```
In [112]: sns.heatmap(corrmatrix,cmap='coolwarm',fmt='.1f',annot=True)
```

```
Out[112]: <Axes: >
```



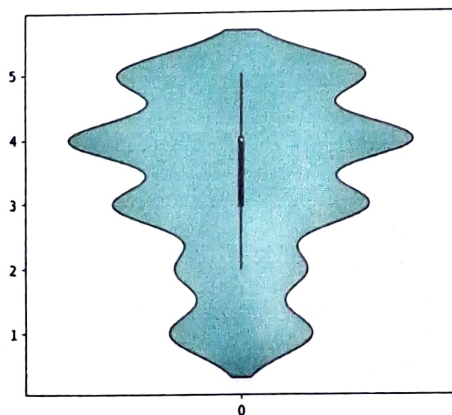
```
In [118]: sns.pairplot(df[['Price','Discount']],diag_kind='kde')
```

```
Out[118]: <seaborn.axisgrid.PairGrid at 0x1e558d5bd90>
```



```
In [122]: sns.violinplot(df['Rating'],palette='Set2')
```

```
Out[122]: <Axes: >
```



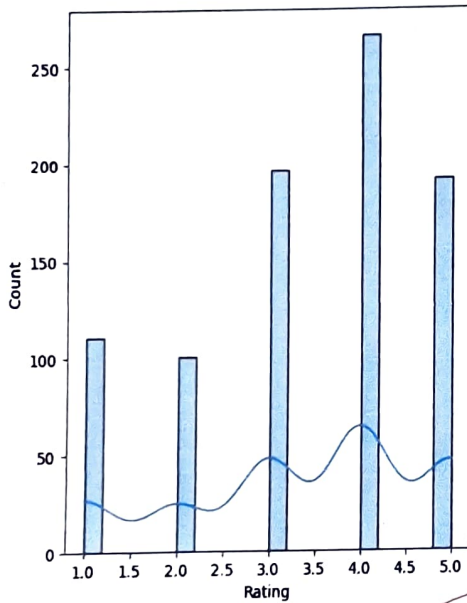
```
In [124]: features=['Price','CustomerAge','Discount','Rating']  
for feature in features:  
    plt.figure(figsize=(5,6))  
    sns.histplot(df[feature],kde=True,bins=20)
```

```
Out[124]: <Axes: xlabel='Rating', ylabel='Count'>
```

```
<Figure size 500x600 with 0 Axes>
```

```
<Figure size 500x600 with 0 Axes>
```

```
<Figure size 500x600 with 0 Axes>
```



BY

R.SHALINI

142224243082

AI DS - 2